Artificial Intelligence

(part 4b)

Exhaustive Search Algorithm

(Exhaustive ..Uninformed..Blind search)





Course Contents

Again..Selected topics for our course. Covering all of AI is impossible!

Key topics include:

- Introduction to Artificial Intelligence (AI)
- Knowledge Representation and Search
- Introduction to AI Programming
- Problem Solving Using Search
- Exhaustive Search Algorithm
- Heuristic Search
- Techniques and Mechanisms of Search Algorithm
- Knowledge Representation Issues and Concepts
- Strong Method Problem Solving
- Reasoning in Uncertain Situations
- Soft Computing and Machine Learning

Selection of a Search Strategy

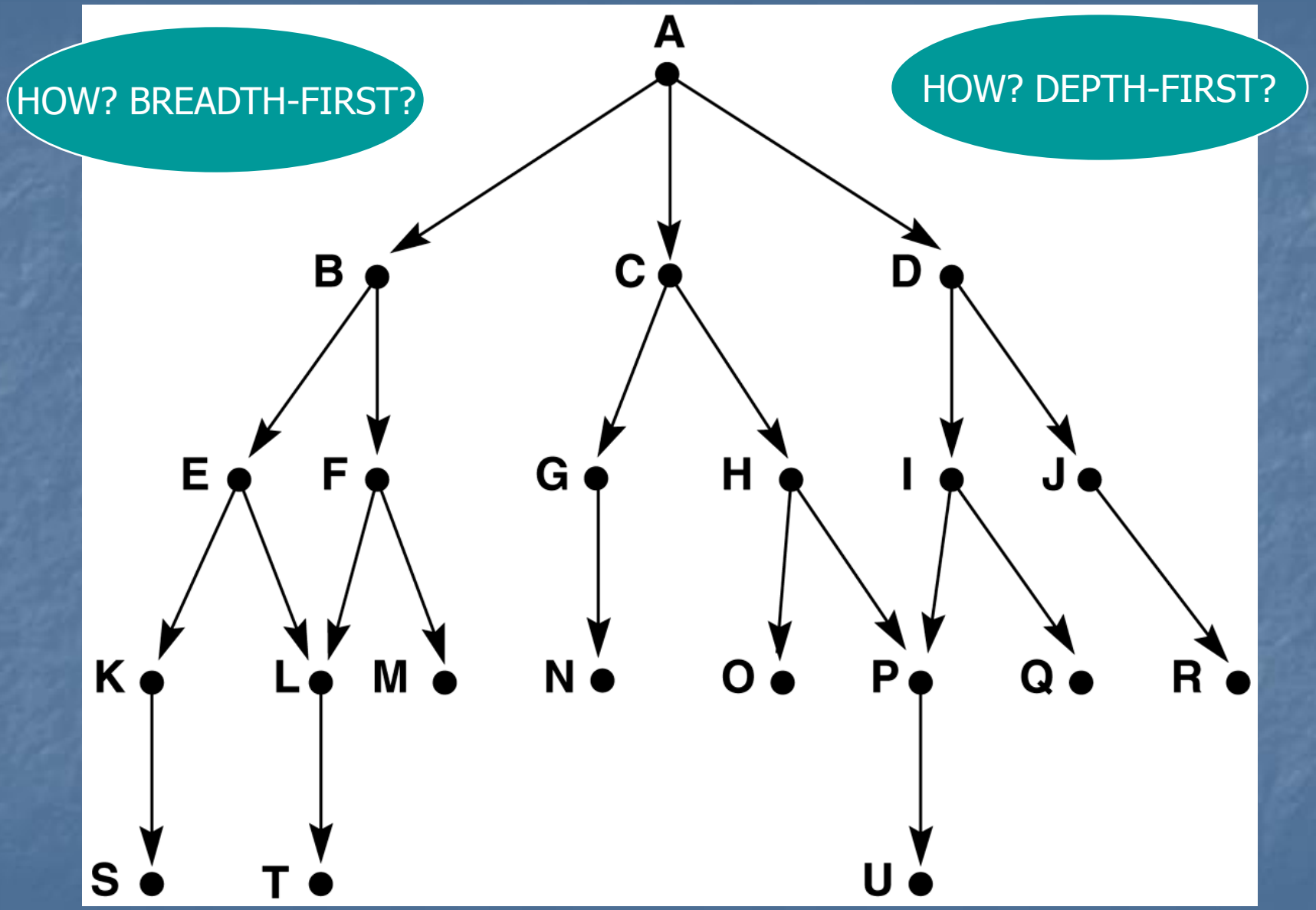
- most of the effort is often spent on the selection of an appropriate search strategy for a given problem
  - uninformed search (blind..exhaustive search)
    - number of steps, path cost unknown
    - knows when it reaches a goal
  - informed search (heuristic search)
    - has background information about the problem
      - map, costs of actions

Search Strategies (The Order..)

- Uninformed Search
  - breadth-first
  - depth-first
  - iterative deepening
  - uniform-cost search
  - depth-limited search
  - bi-directional search
  - constraint satisfaction

- Informed Search
  - best-first search
  - search with heuristics
  - memory-bounded search
  - iterative improvement search

Graph for breadth- and depth-first search examples.



Breadth-First

- all the nodes reachable from the current node are explored first
  - achieved by the TREE-SEARCH method by appending newly generated nodes at the end of the search queue

```
function BREADTH-FIRST-SEARCH(problem) returns solution

    return TREE-SEARCH(problem, FIFO-QUEUE())
```

Time Complexity	b^{d+1}
Space Complexity	b^{d+1}
Completeness	yes (for finite b)
Optimality	yes (for non-negative path costs)

b	branching factor
d	depth of the tree

Solution is found

Function breadth_ first search algorithm

```
function breadth_first_search;

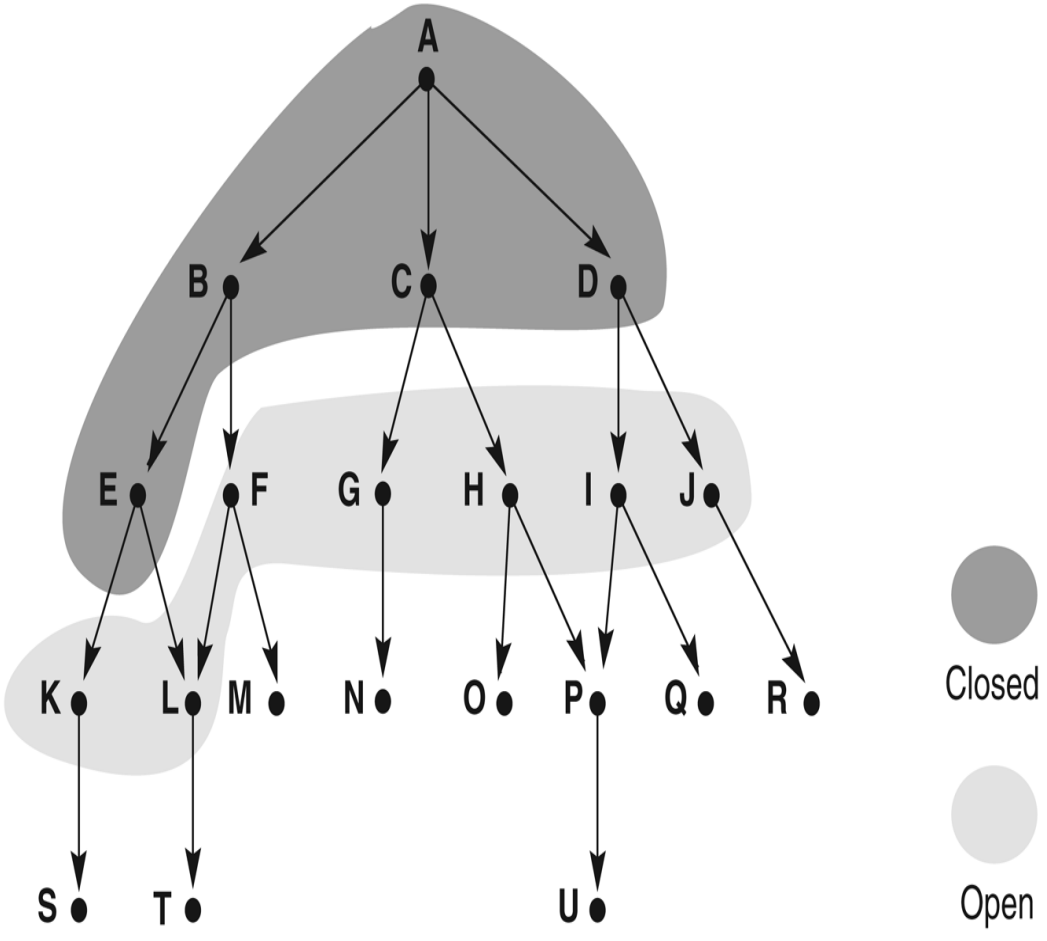
begin
   open := [Start];                                                    % initialize
   closed := [ ];
   while open ≠ [ ] do                                                 % states remain
      begin
         remove leftmost state from open, call it X;
            if X is a goal then return SUCCESS                        % goal found
               else begin
                  generate children of X;
                  put X on closed;
                  discard children of X if already on open or closed;  % loop check
                  put remaining children on right end of open          % queue
               end
      end
   return FAIL                                                         % no states left
end.
```

1. **open = [A]; closed = [ ]**
2. **open = [B,C,D]; closed = [A]**
3. **open = [C,D,E,F]; closed = [B,A]**
4. **open = [D,E,F,G,H]; closed = [C,B,A]**
5. **open = [E,F,G,H,I,J]; closed = [D,C,B,A]**
6. **open = [F,G,H,I,J,K,L]; closed = [E,D,C,B,A]**
7. **open = [G,H,I,J,K,L,M]** (as L is already on open); **closed = [F,E,D,C,B,A]**
8. **open = [H,I,J,K,L,M,N]; closed = [G,F,E,D,C,B,A]**
9. and so on until either U is found or **open** = [ ]

breadth-first search

- States on open and closed are highlighted.
- Explore level by level
- Use list open ~ NSL in backtrack – states not yet evaluated
- Use list close ~ SL and DE in backtrack –states already evaluated
- Which state is removed from open determines the order of search- BFS adds at the right list and removes from the left (queue-FIFO)

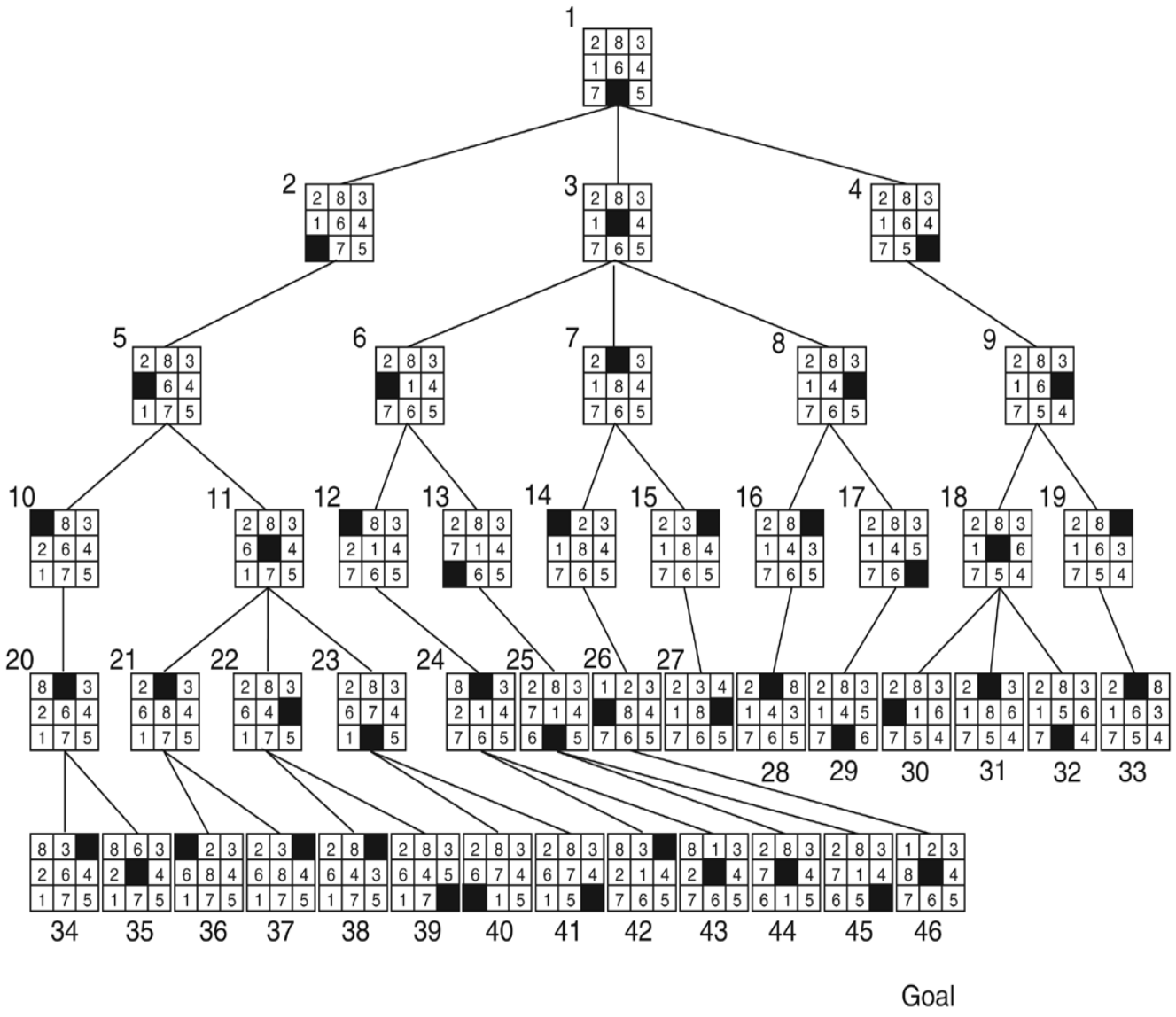
1. **open = [A]; closed = [ ]**
2. **open = [B,C,D]; closed = [A]**
3. **open = [C,D,E,F]; closed = [B,A]**
4. **open = [D,E,F,G,H]; closed = [C,B,A]**
5. **open = [E,F,G,H,I,J]; closed = [D,C,B,A]**
6. **open = [F,G,H,I,J,K,L]; closed = [E,D,C,B,A]**
7. **open = [G,H,I,J,K,L,M]** (as L is already on open); **closed = [F,E,D,C**
8. **open = [H,I,J,K,L,M,N]; closed = [G,F,E,D,C,B,A]**
9. and so on until either U is found or **open** = [ ]



```
function breadth_first_search;

begin
  open := [Start];                                  % initialize
  closed := [ ];
  while open ≠ [ ] do                               % states remain
    begin
      remove leftmost state from open, call it X;
        if X is a goal then return SUCCESS
          else begin
            generate children of X;
            put X on closed;
            discard children of X if already on open or closed;
            put remaining children on right end of open
          end
    end
  return FAIL
end.
```

Breadth-first search of the 8-puzzle, showing order in which states were removed from open.



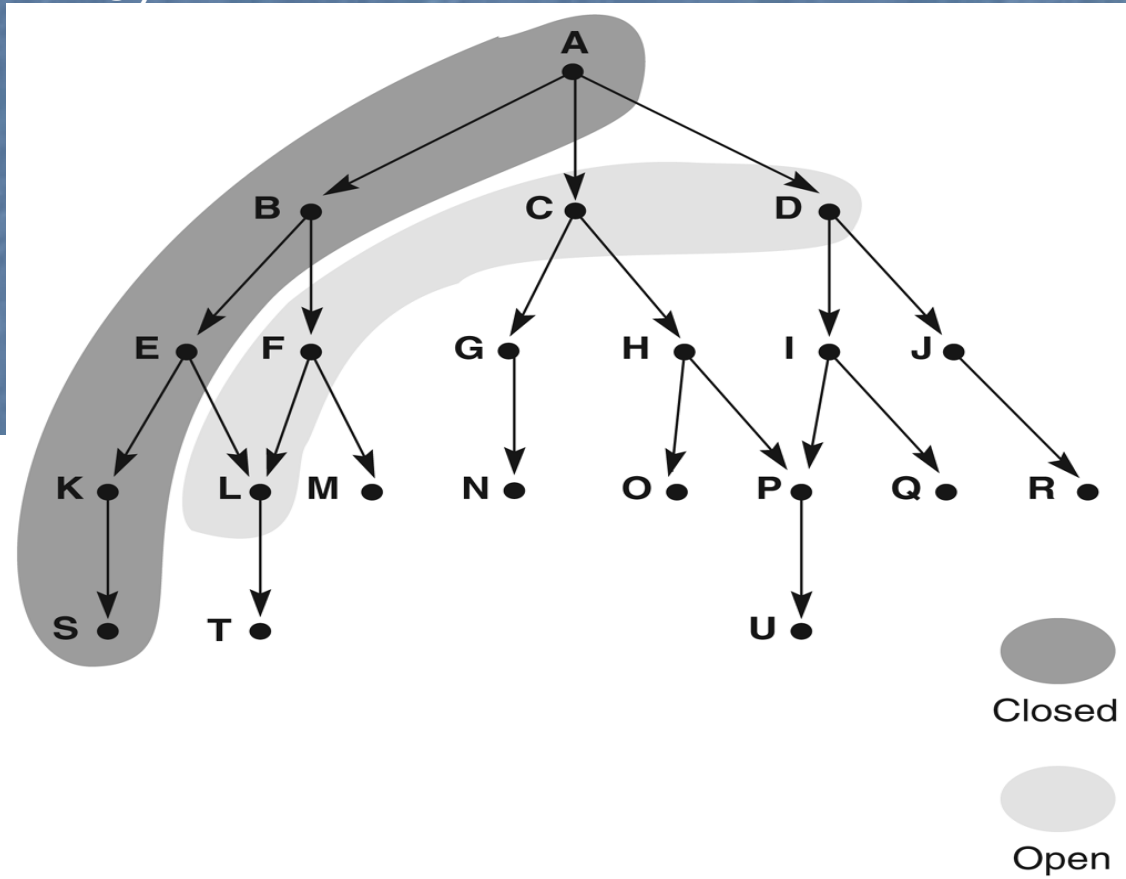
Function depth_first_search algorithm and the trace

```
begin
   open := [Start];                                                    % initialize
   closed := [ ];
   while open ≠ [ ] do                                                 % states remain
      begin
         remove leftmost state from open, call it X;
         if X is a goal then return SUCCESS                            % goal found
            else begin
               generate children of X;
               put X on closed;
               discard children of X if already on open or closed;     % loop check
               put remaining children on left end of open              % stack
            end
      end;
   return FAIL                                                         % no states left
end.
```

1. open = [A]; closed = [ ]
2. open = [B,C,D]; closed = [A]
3. open = [E,F,C,D]; closed = [B,A]
4. open = [K,L,F,C,D]; closed = [E,B,A]
5. open = [S,L,F,C,D]; closed = [K,E,B,A]
6. open = [L,F,C,D]; closed = [S,K,E,B,A]
7. open = [T,F,C,D]; closed = [L,S,K,E,B,A]
8. open = [F,C,D]; closed = [T,L,S,K,E,B,A]
9. open = [M,C,D], as L is already on closed; closed = [F,T,L,S,K,E,B,A]
10. open = [C,D]; closed = [M,F,T,L,S,K,E,B,A]
11. open = [G,H,D]; closed = [C,M,F,T,L,S,K,E,B,A]

Depth-first search (DFS)

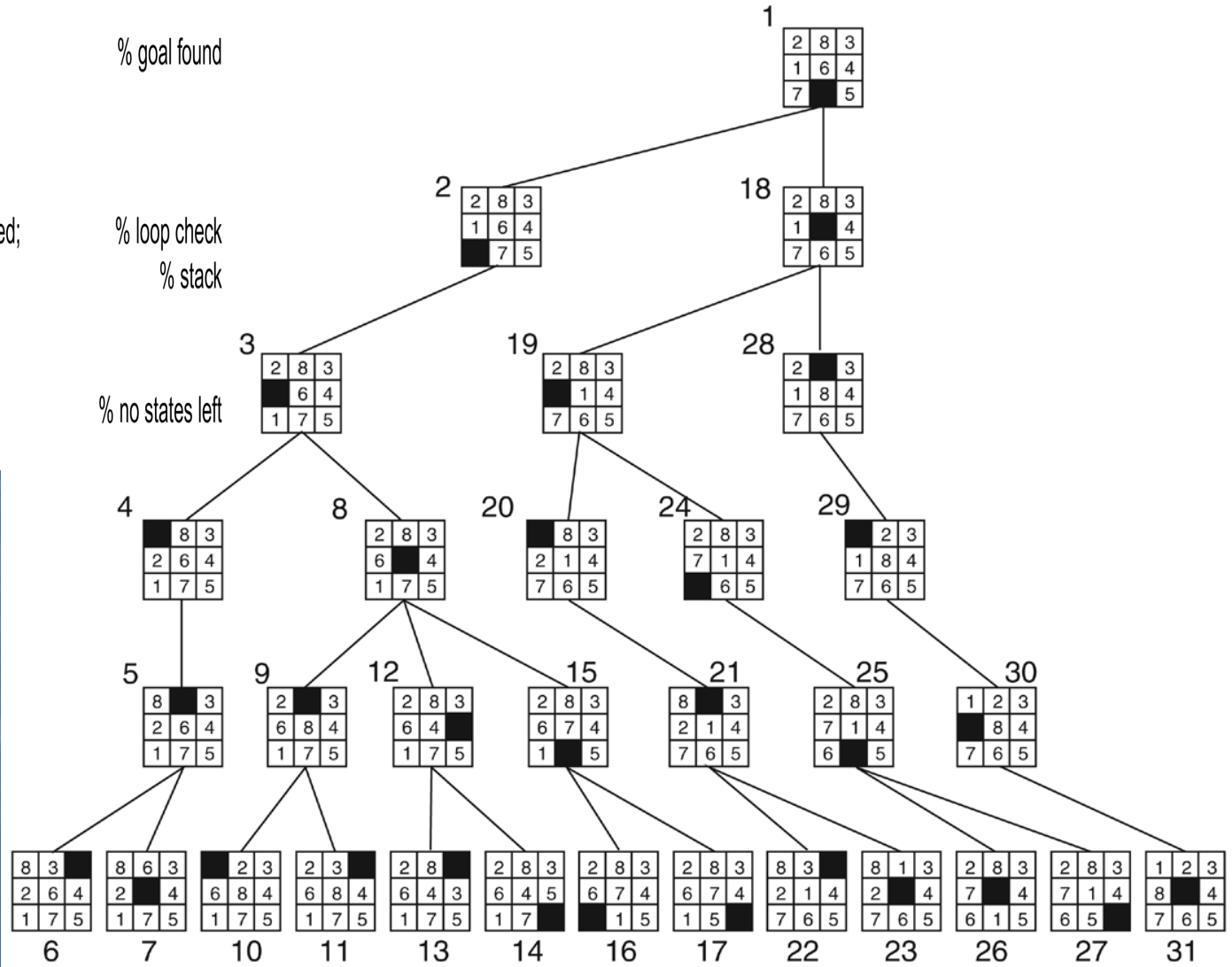
- States on open and closed are highlighted.
- Child and descendents are evaluated before siblings
- Goes deeper into search space whenever possible
- Which state is removed from open determines the order of search- DFS adds and removes from the left end (stack-LIFO)



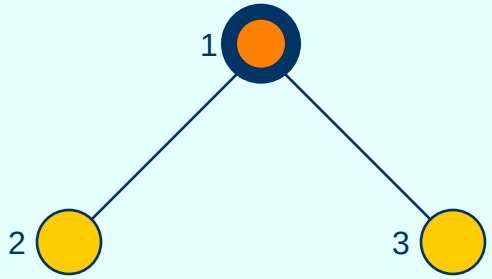
1. open = [A]; closed = [ ]
2. open = [B,C,D]; closed = [A]
3. open = [E,F,C,D]; closed = [B,A]
4. open = [K,L,F,C,D]; closed = [E,B,A]
5. open = [S,L,F,C,D]; closed = [K,E,B,A]
6. open = [L,F,C,D]; closed = [S,K,E,B,A]
7. open = [T,F,C,D]; closed = [L,S,K,E,B,A]
8. open = [F,C,D]; closed = [T,L,S,K,E,B,A]
9. open = [M,C,D], as L is already on closed; closed = [F,T,L,S,K,E,B,A]
10. open = [C,D]; closed = [M,F,T,L,S,K,E,B,A]
11. open = [G,H,D]; closed = [C,M,F,T,L,S,K,E,B,A]

Depth-first search of the 8-puzzle with a depth bound of 5.

```
begin
  open := [Start];                                          % initialize
  closed := [ ];
  while open ≠ [ ] do                                       % states remain
    begin
      remove leftmost state from open, call it X;
      if X is a goal then return SUCCESS                    % goal found
        else begin
          generate children of X;
          put X on closed;
          discard children of X if already on open or closed;   % loop check
          put remaining children on left end of open            % stack
        end
    end;
  return FAIL                                               % no states left
end.
```



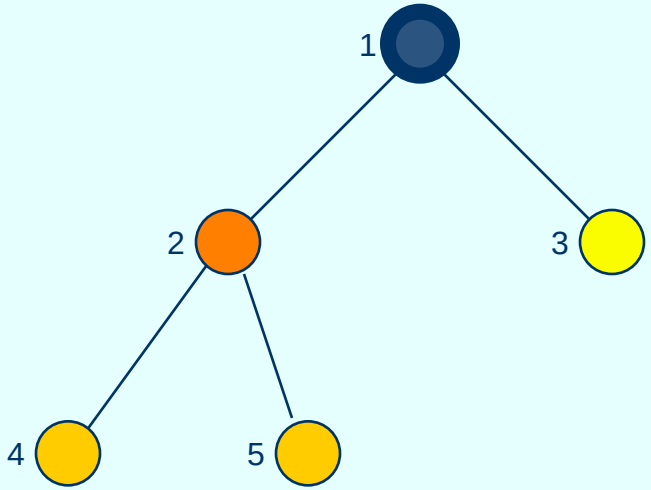
Breadth-First Snapshot 1



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [] + [2,3]

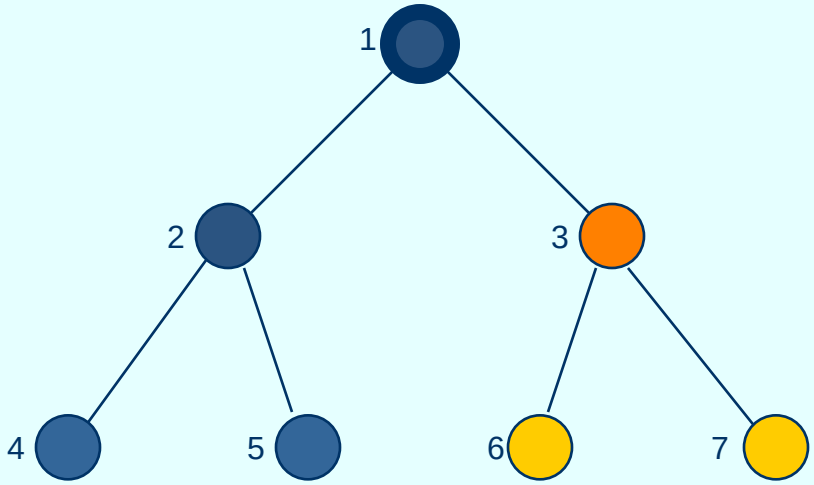
Breadth-First Snapshot 2



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [3] + [4,5]

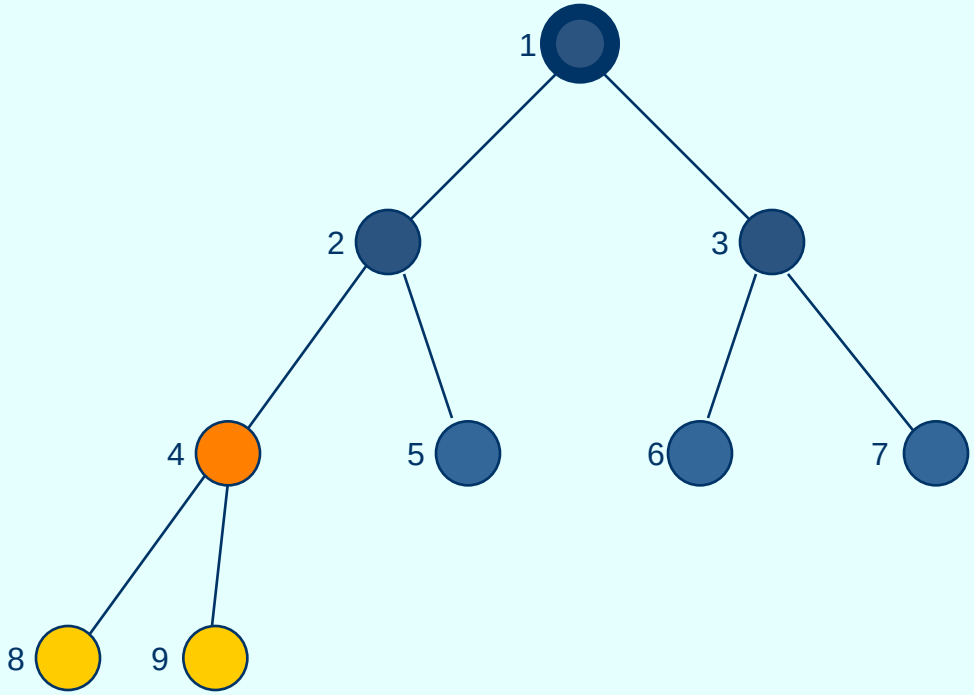
Breadth-First Snapshot 3



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [4,5] + [6,7]

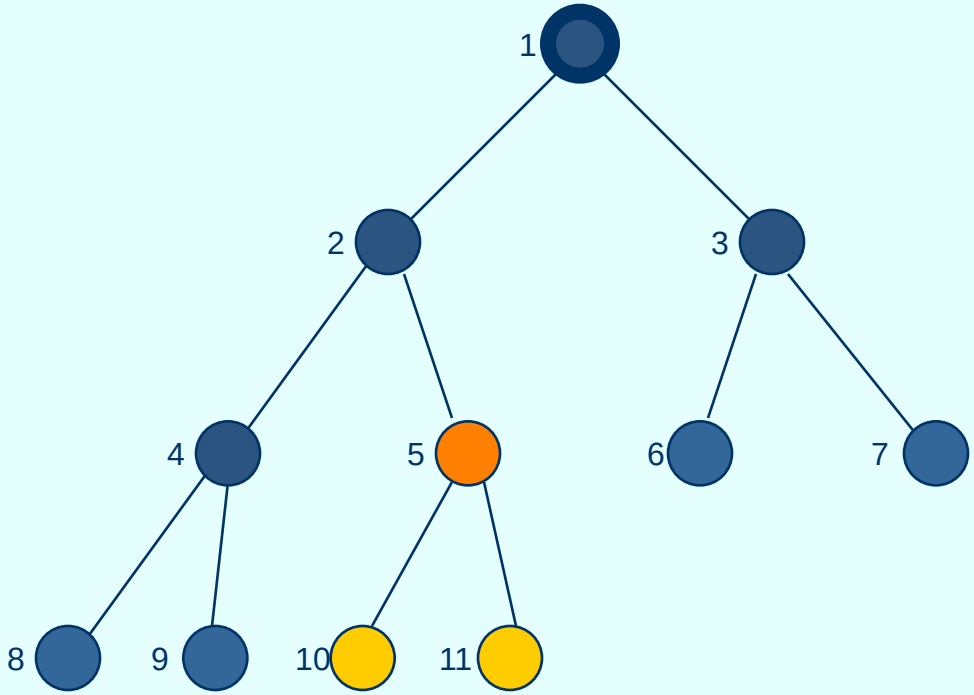
Breadth-First Snapshot 4



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [5,6,7] + [8,9]

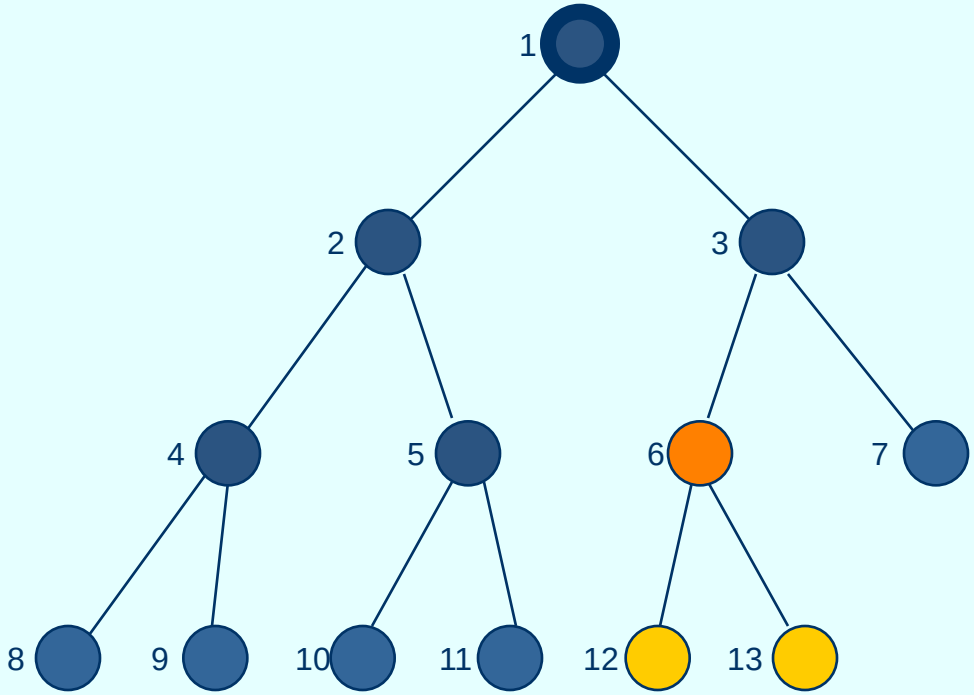
Breadth-First Snapshot 5



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [6,7,8,9] + [10,11]

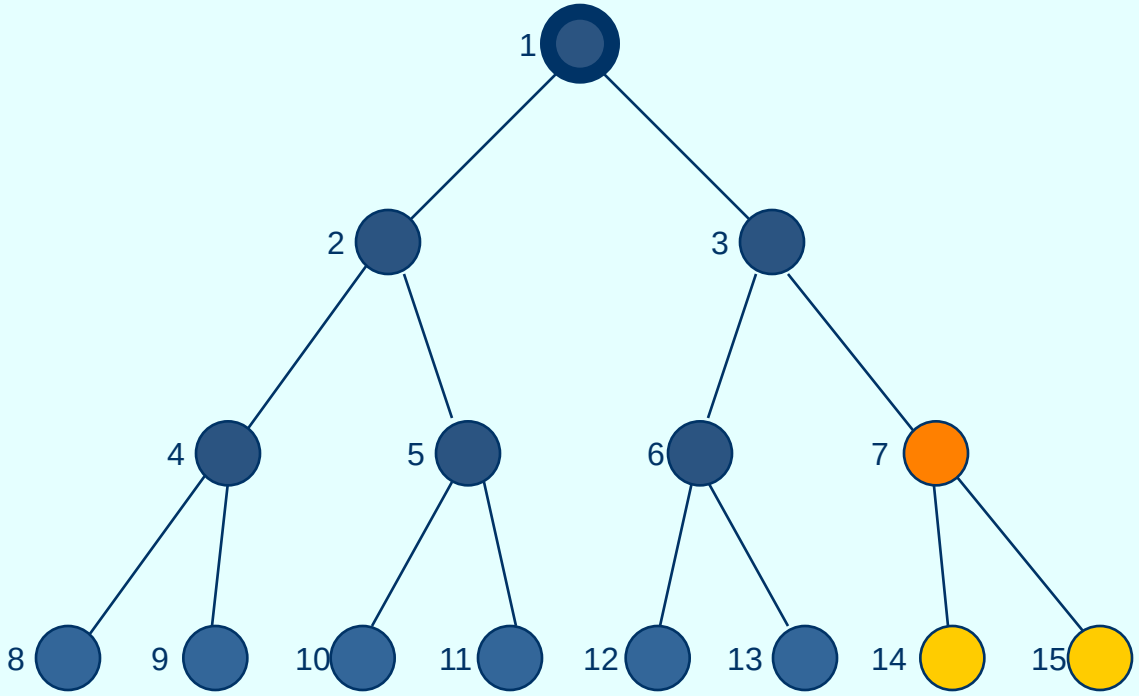
Breadth-First Snapshot 6



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [7,8,9,10,11] + [12,13]

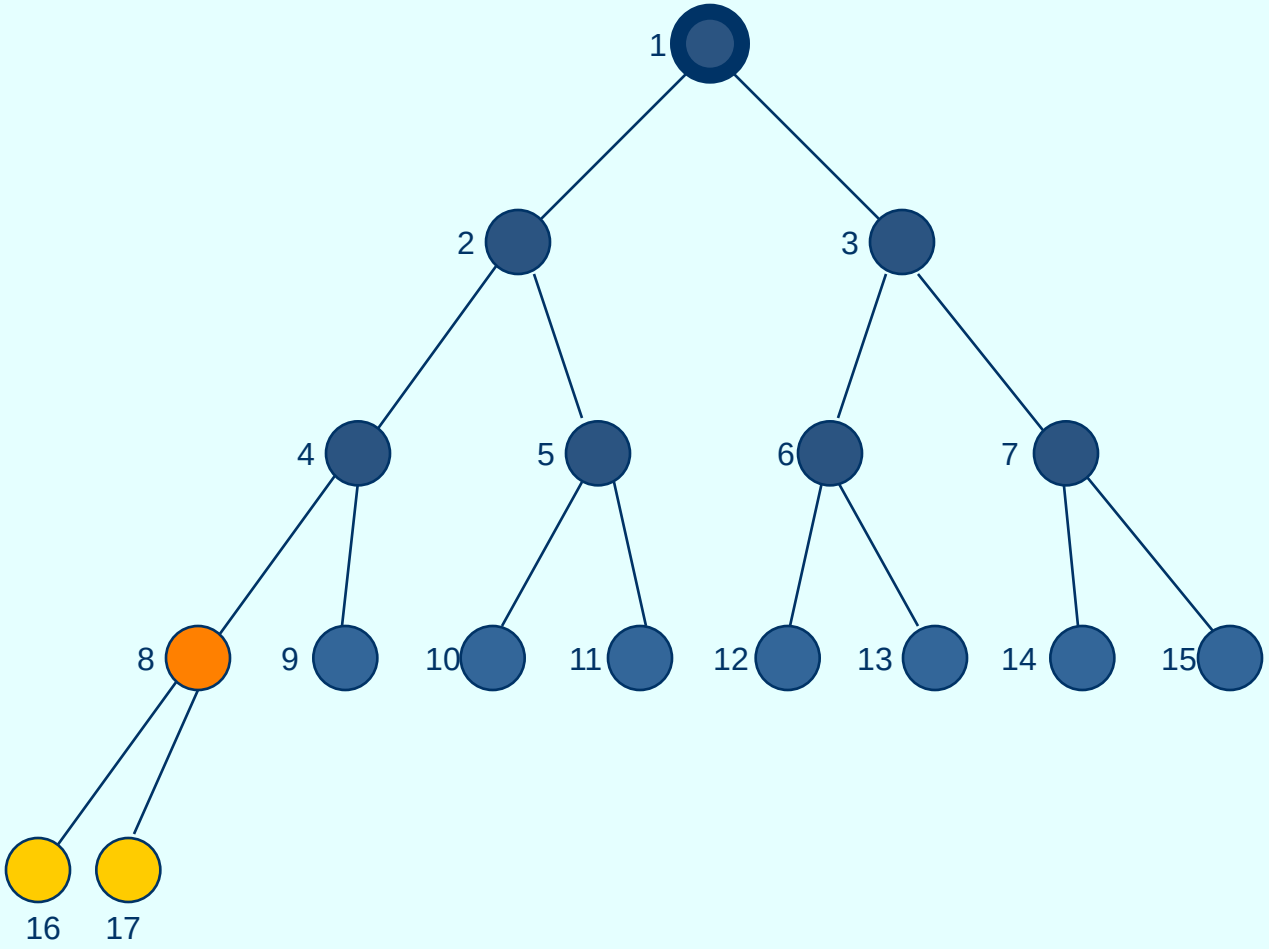
Breadth-First Snapshot 7



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [8,9.10,11,12,13] + [14,15]

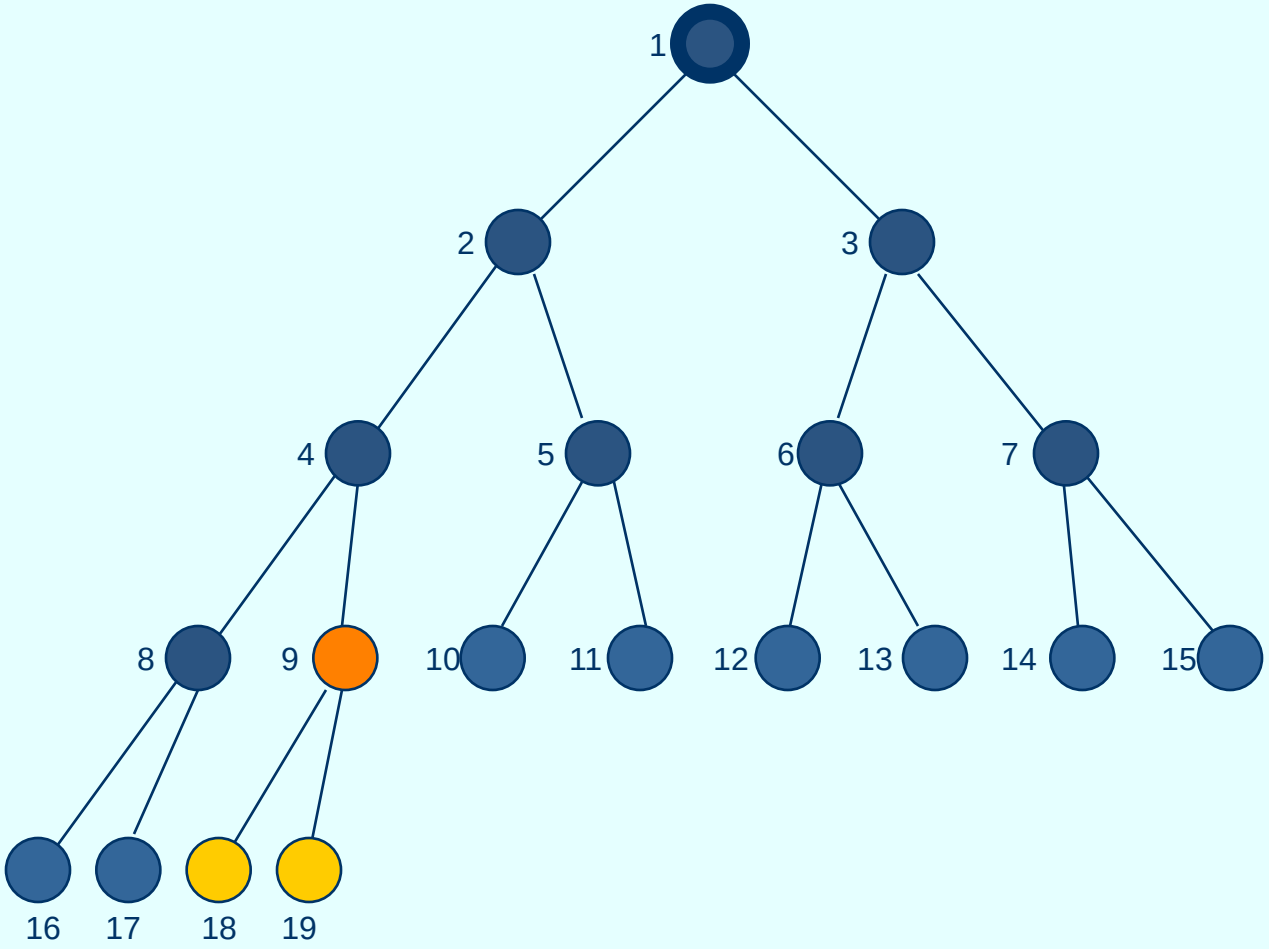
Breadth-First Snapshot 8



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [9,10,11,12,13,14,15] + [16,17]

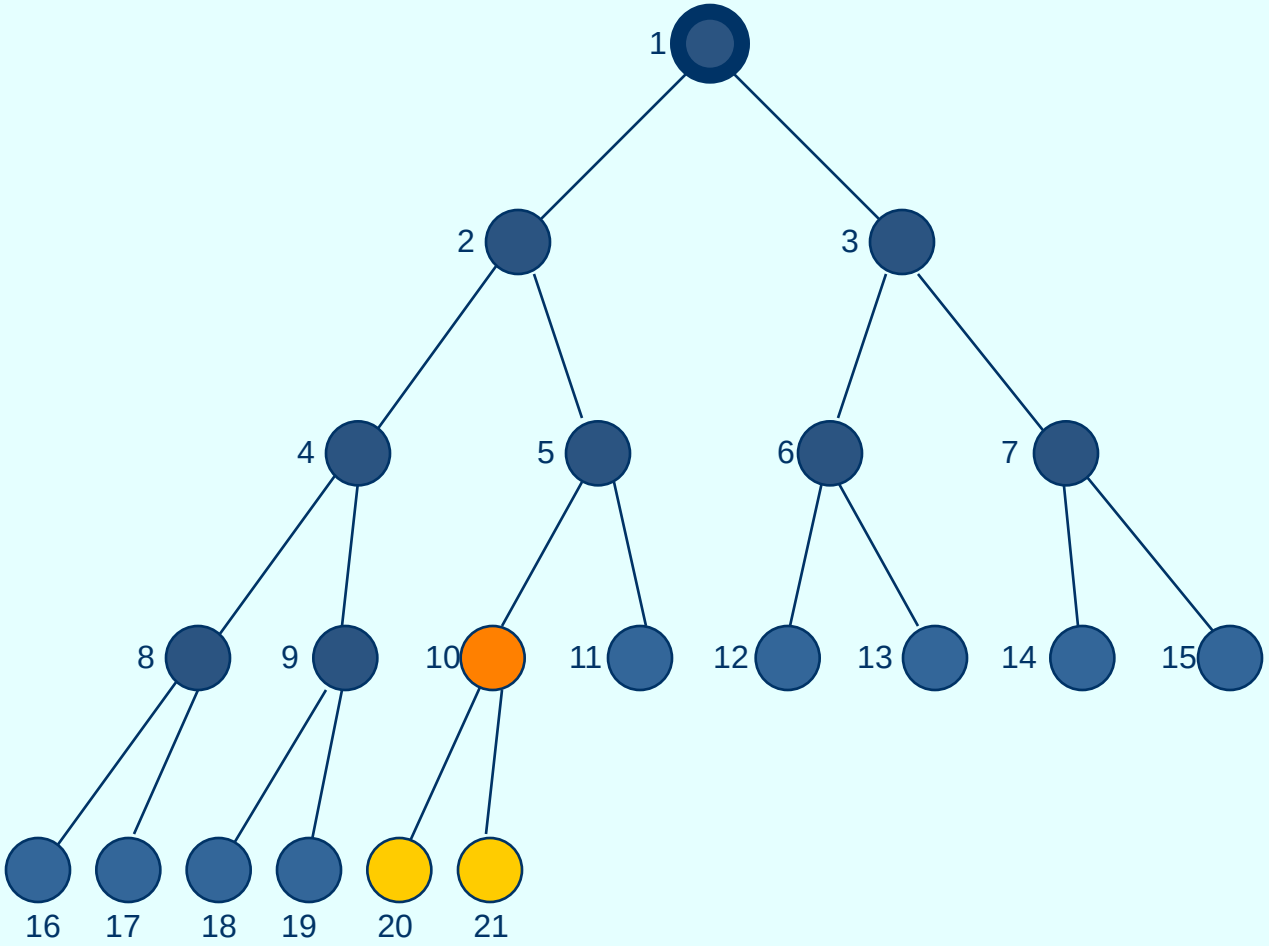
Breadth-First Snapshot 9



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [10,11,12,13,14,15,16,17] + [18,19]

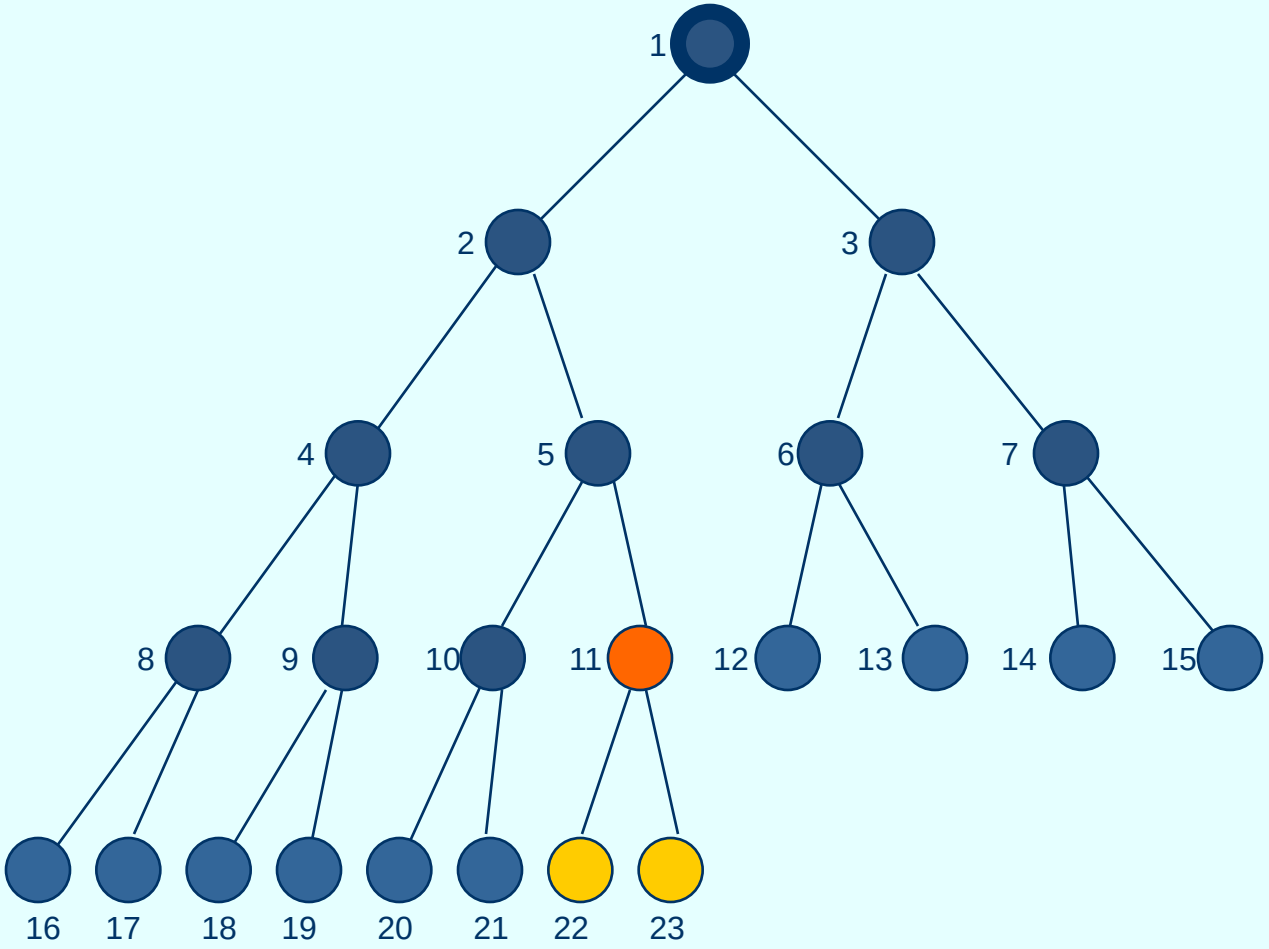
Breadth-First Snapshot 10



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [11,12,13,14,15,16,17,18,19] + [20,21]

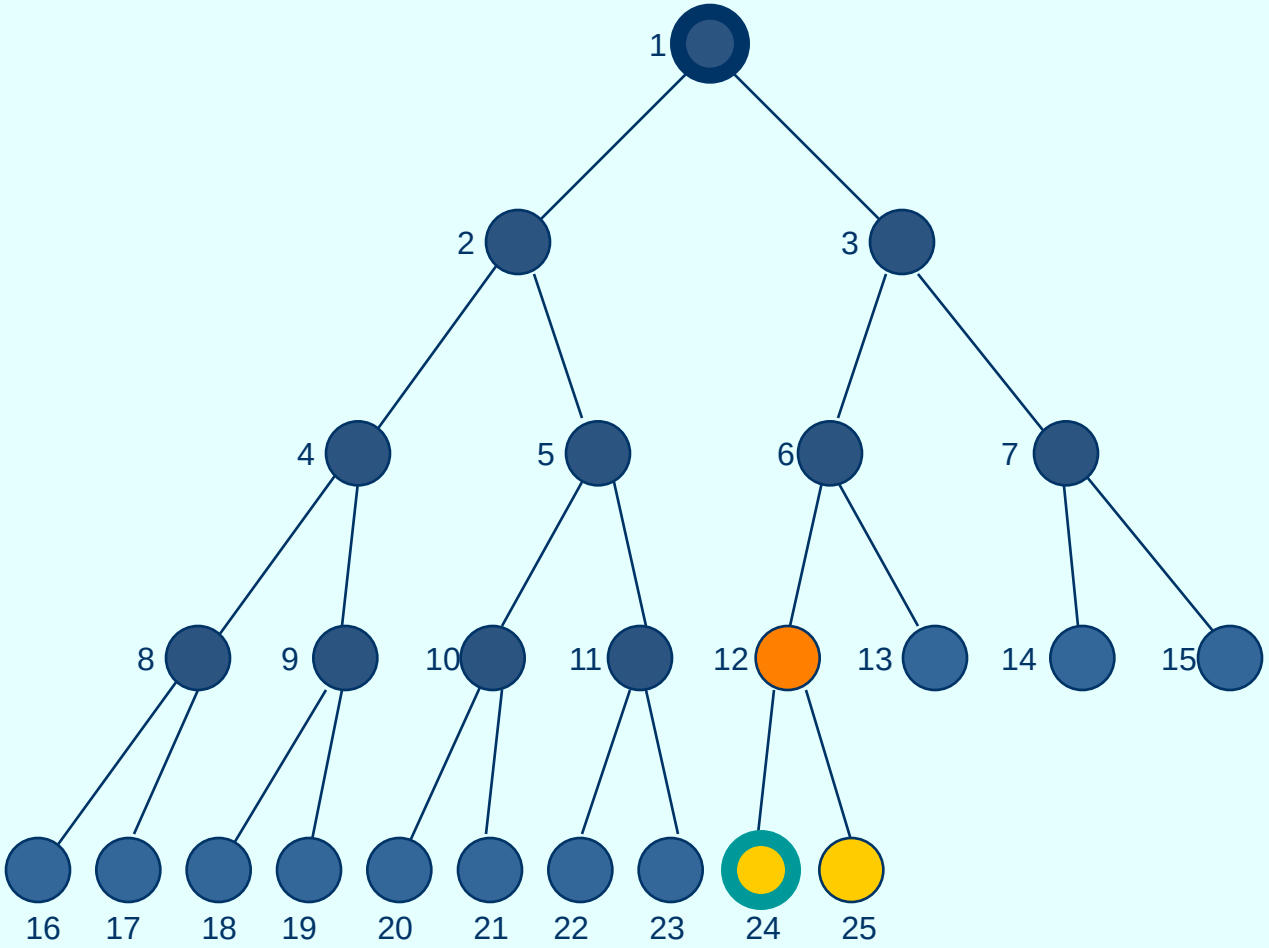
Breadth-First Snapshot 11



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [12, 13, 14, 15, 16, 17, 18, 19, 20, 21] + [22,23]

Breadth-First Snapshot 12

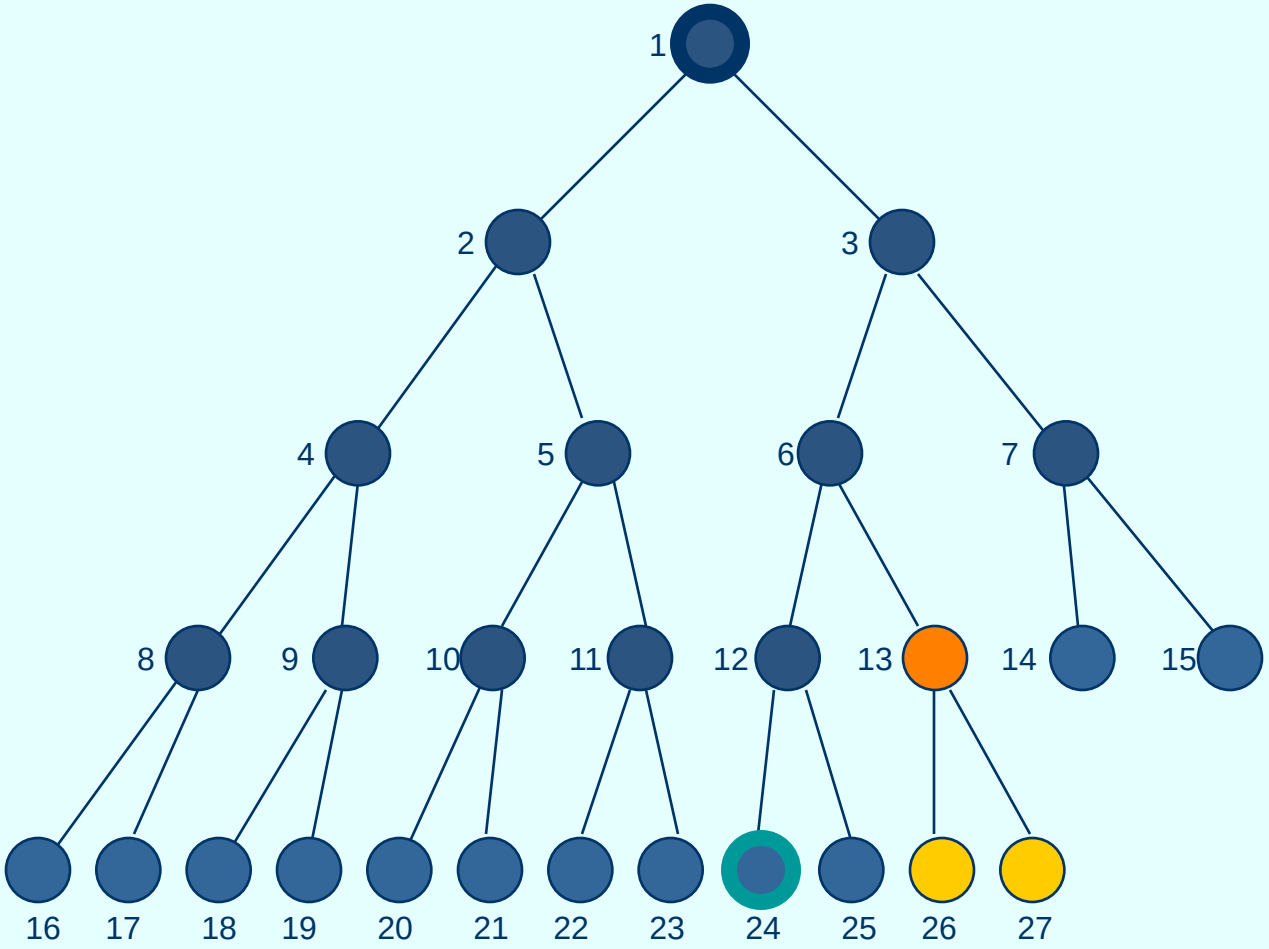


Initial
Visited
Fringe
Current
Visible
Goal

Note:
The goal node is "visible" here, but we can not perform the goal test yet.

Fringe: [13,14,15,16,17,18,19,20,21] + [22,23]

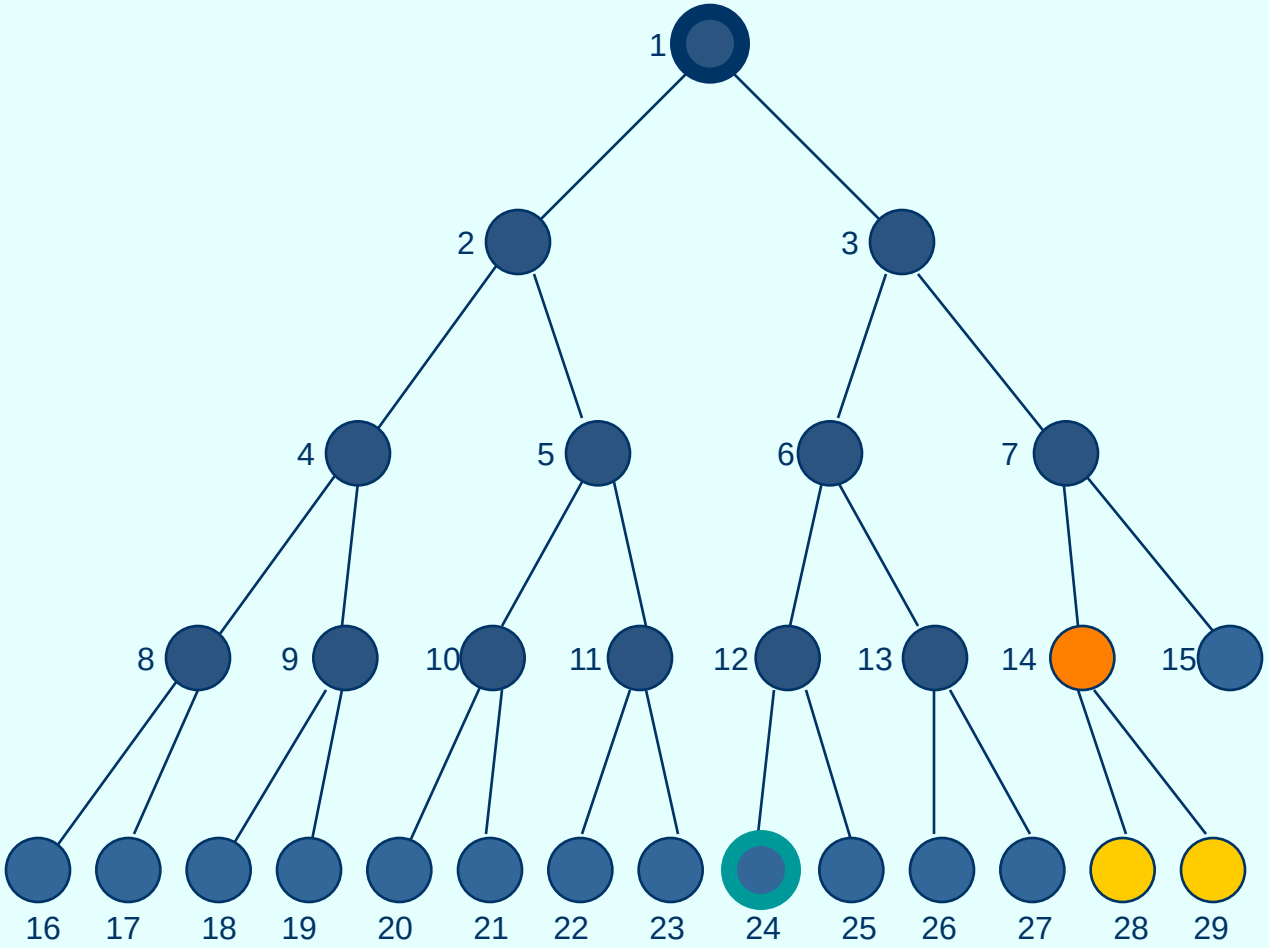
Breadth-First Snapshot 13



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [14,15,16,17,18,19,20,21,22,23,24,25] + [26,27]

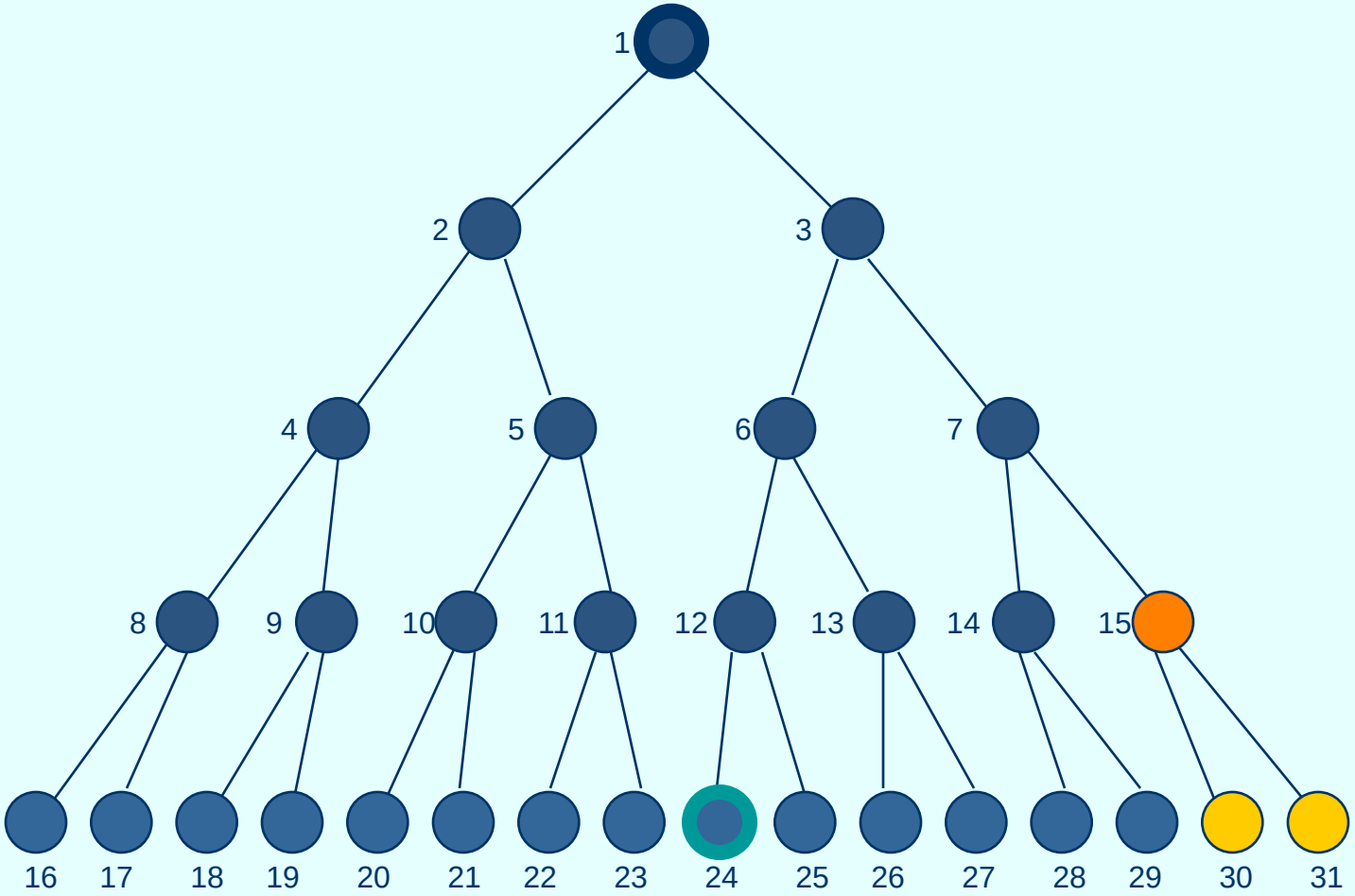
Breadth-First Snapshot 14



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [15,16,17,18,19,20,21,22,23,24,25,26,27] + [28,29]

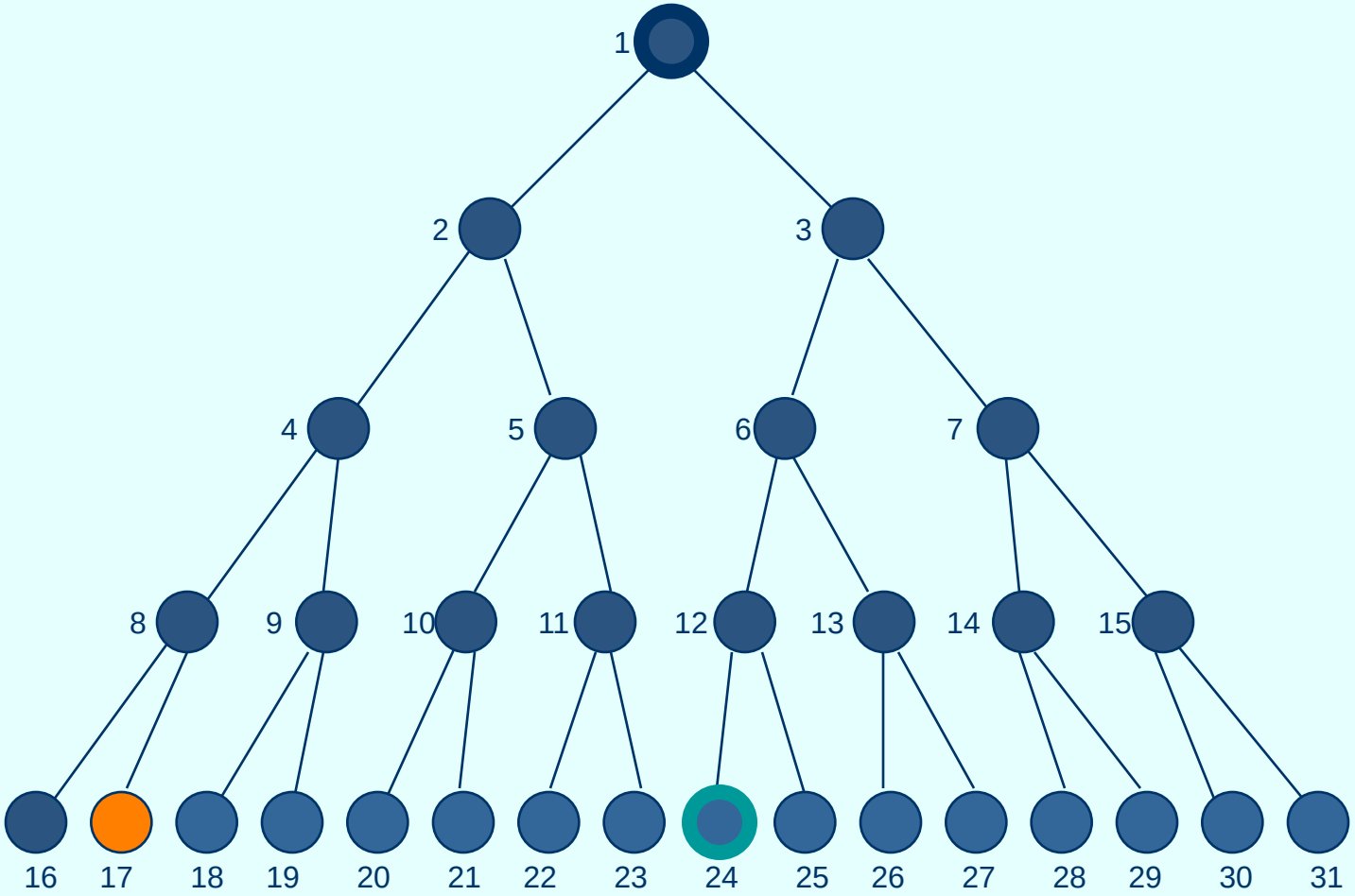
Breadth-First Snapshot 15



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [15,16,17,18,19,20,21,22,23,24,25,26,27,28,29] + [30,31]

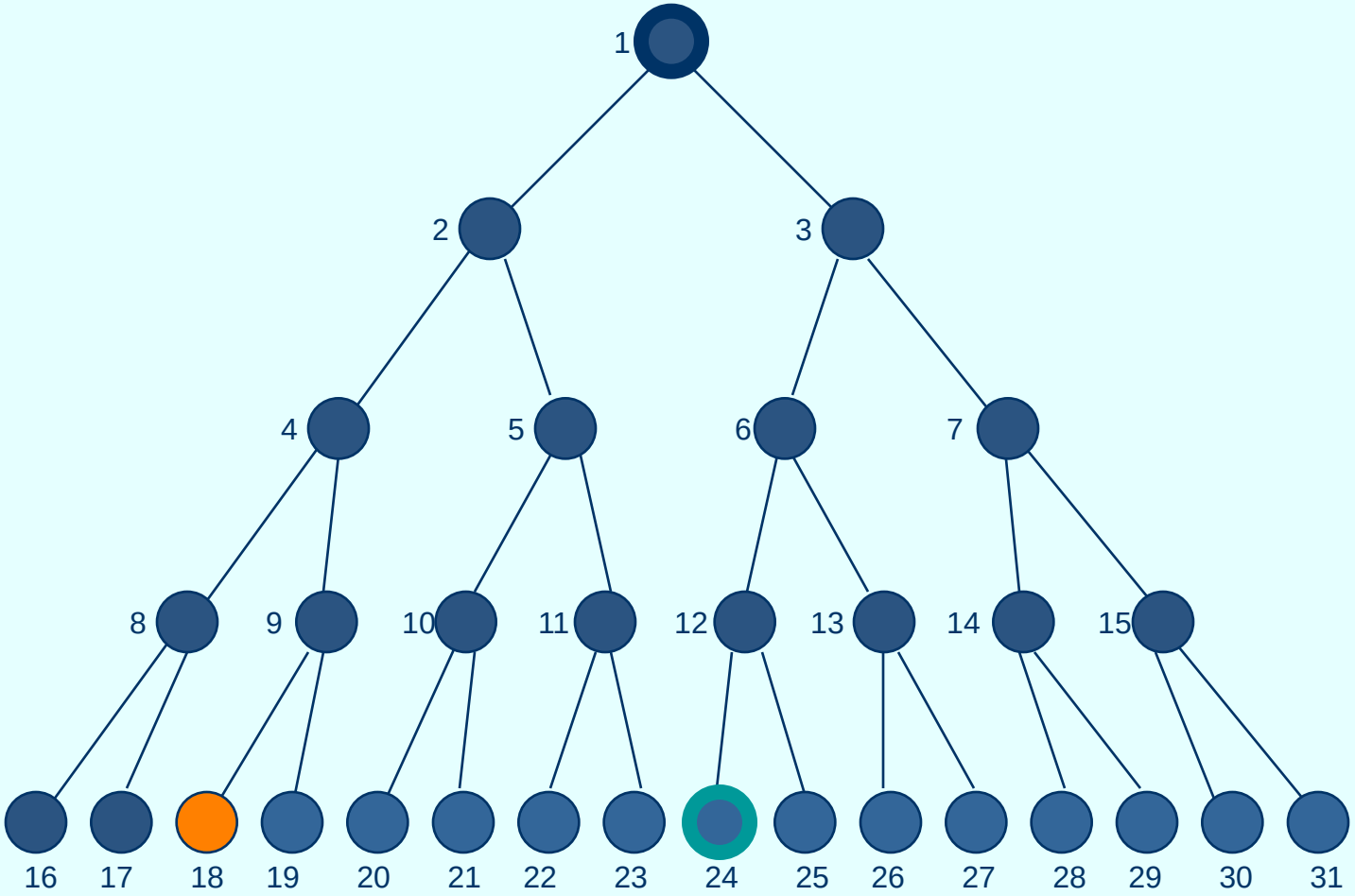
Breadth-First Snapshot 17



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [18,19,20,21,22,23,24,25,26,27,28,29,30,31]

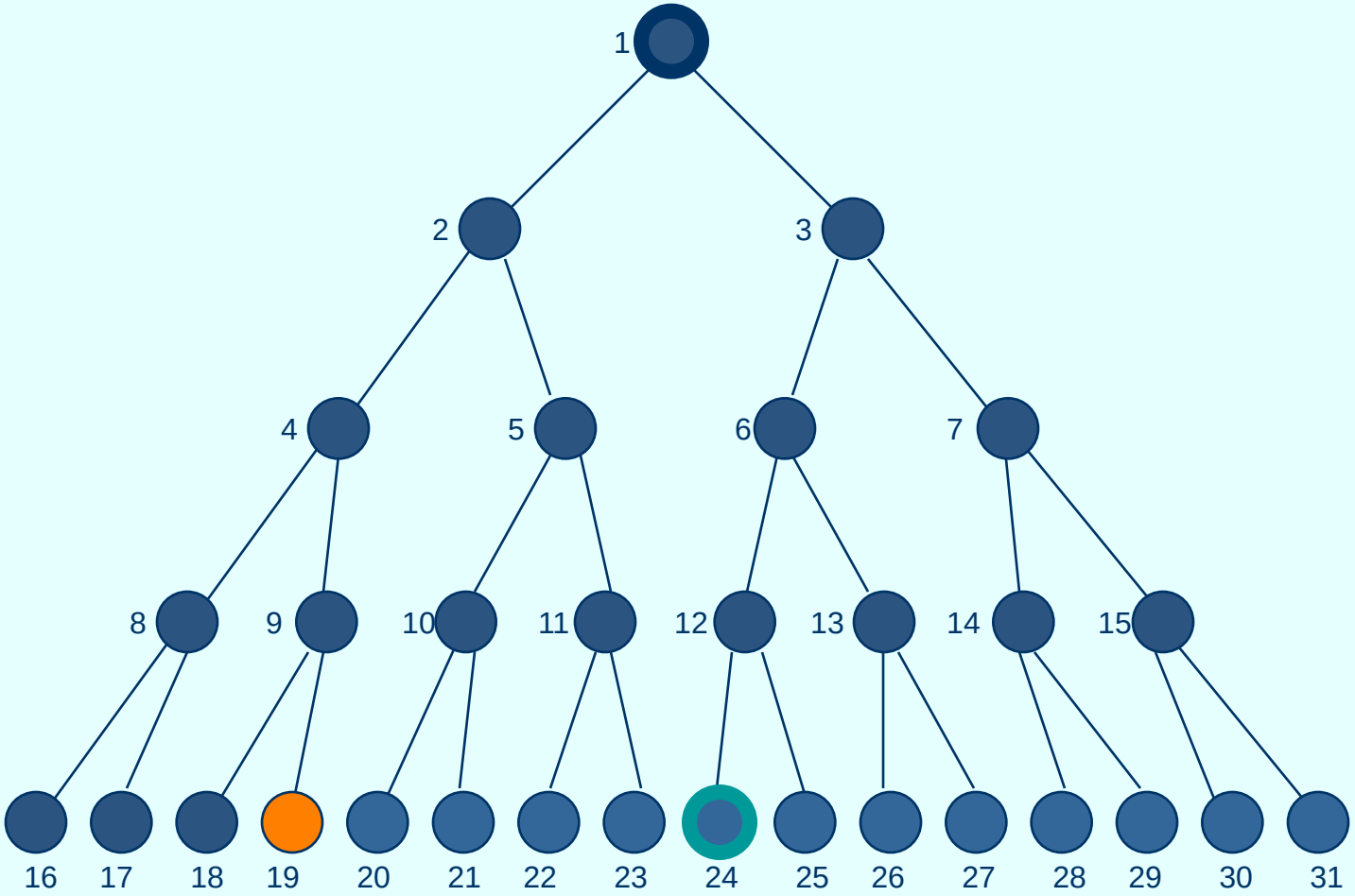
Breadth-First Snapshot 18



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [19,20,21,22,23,24,25,26,27,28,29,30,31]

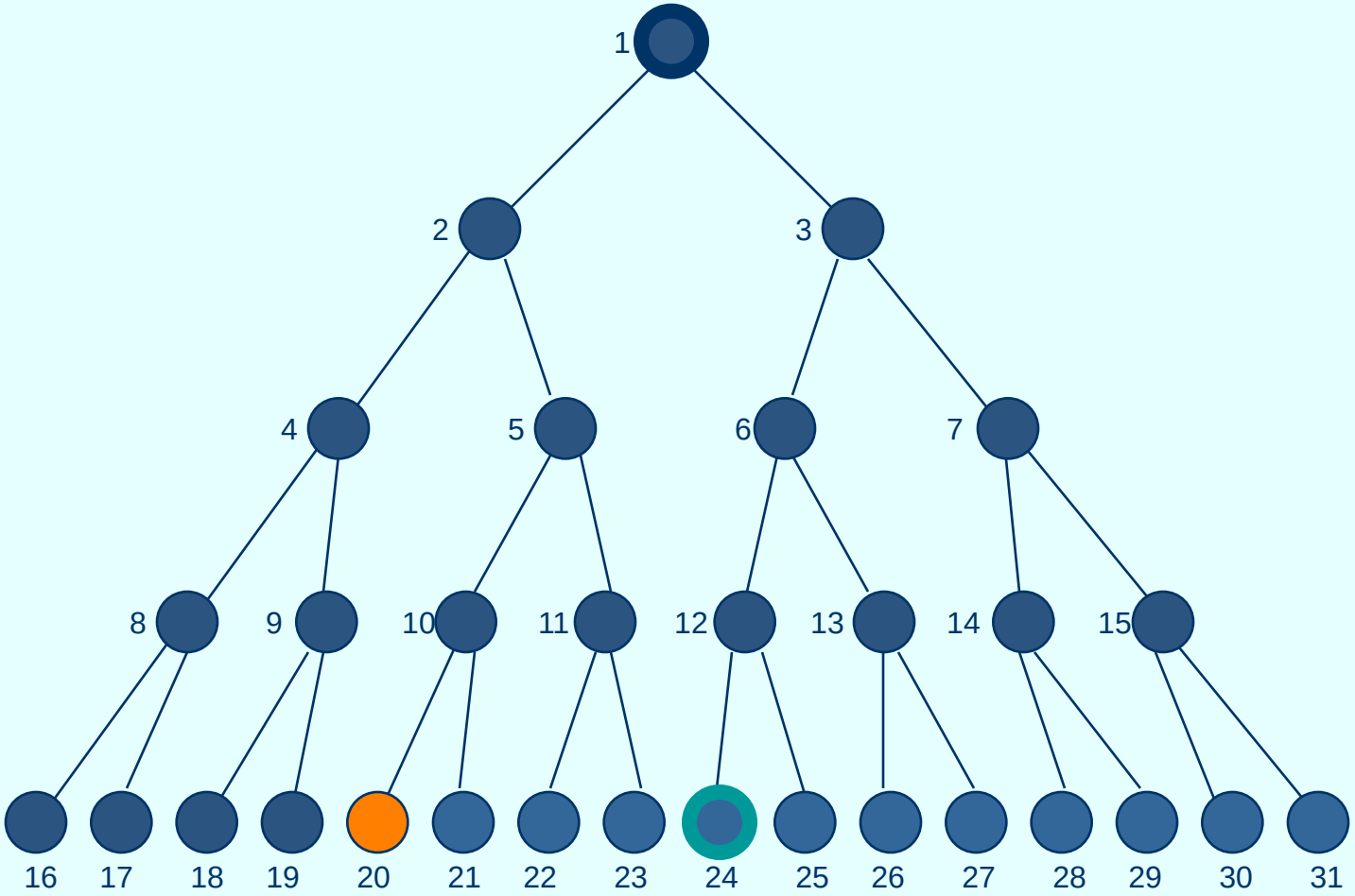
Breadth-First Snapshot 19



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [20,21,22,23,24,25,26,27,28,29,30,31]

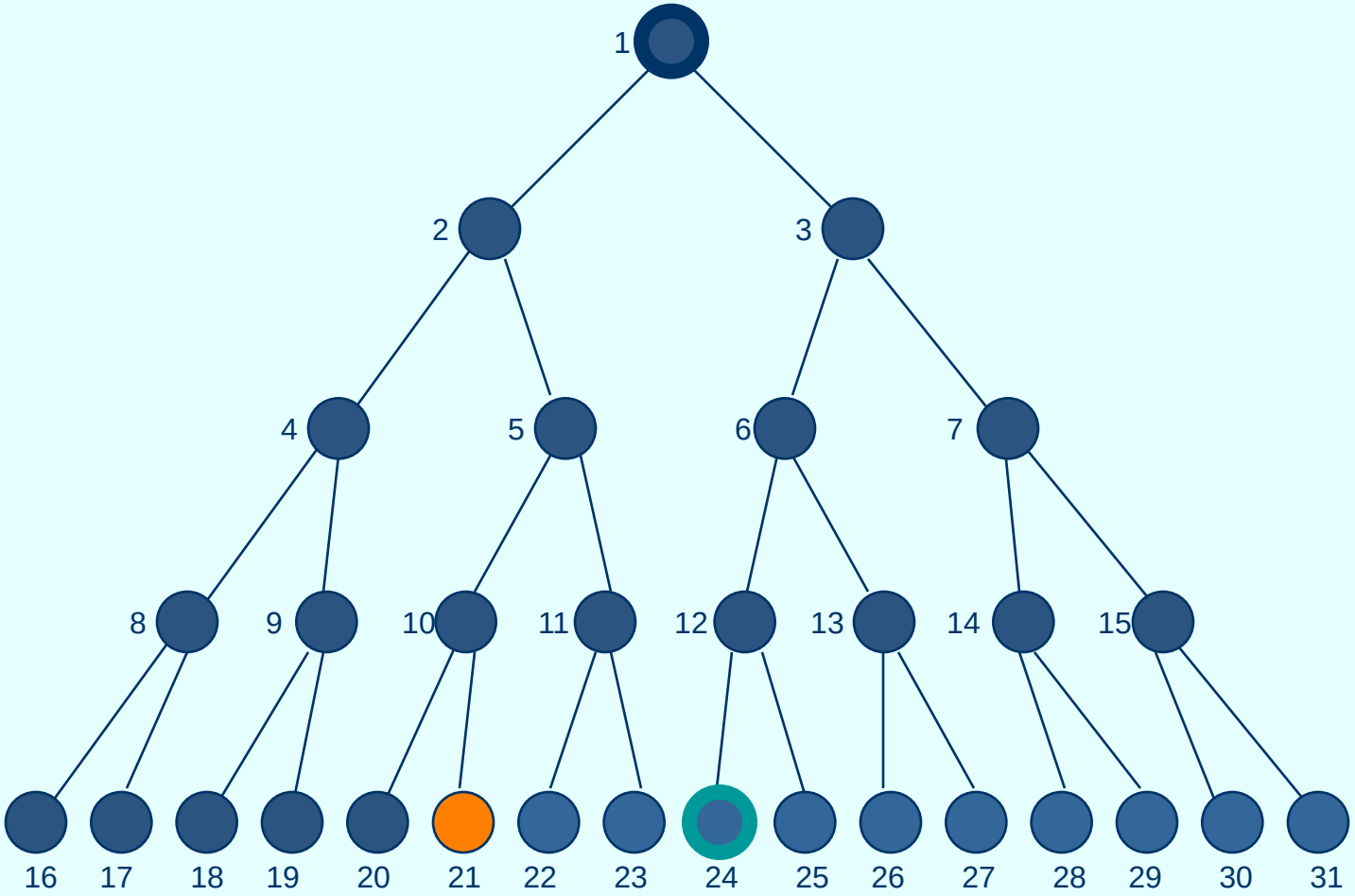
Breadth-First Snapshot 20



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [21,22,23,24,25,26,27,28,29,30,31]

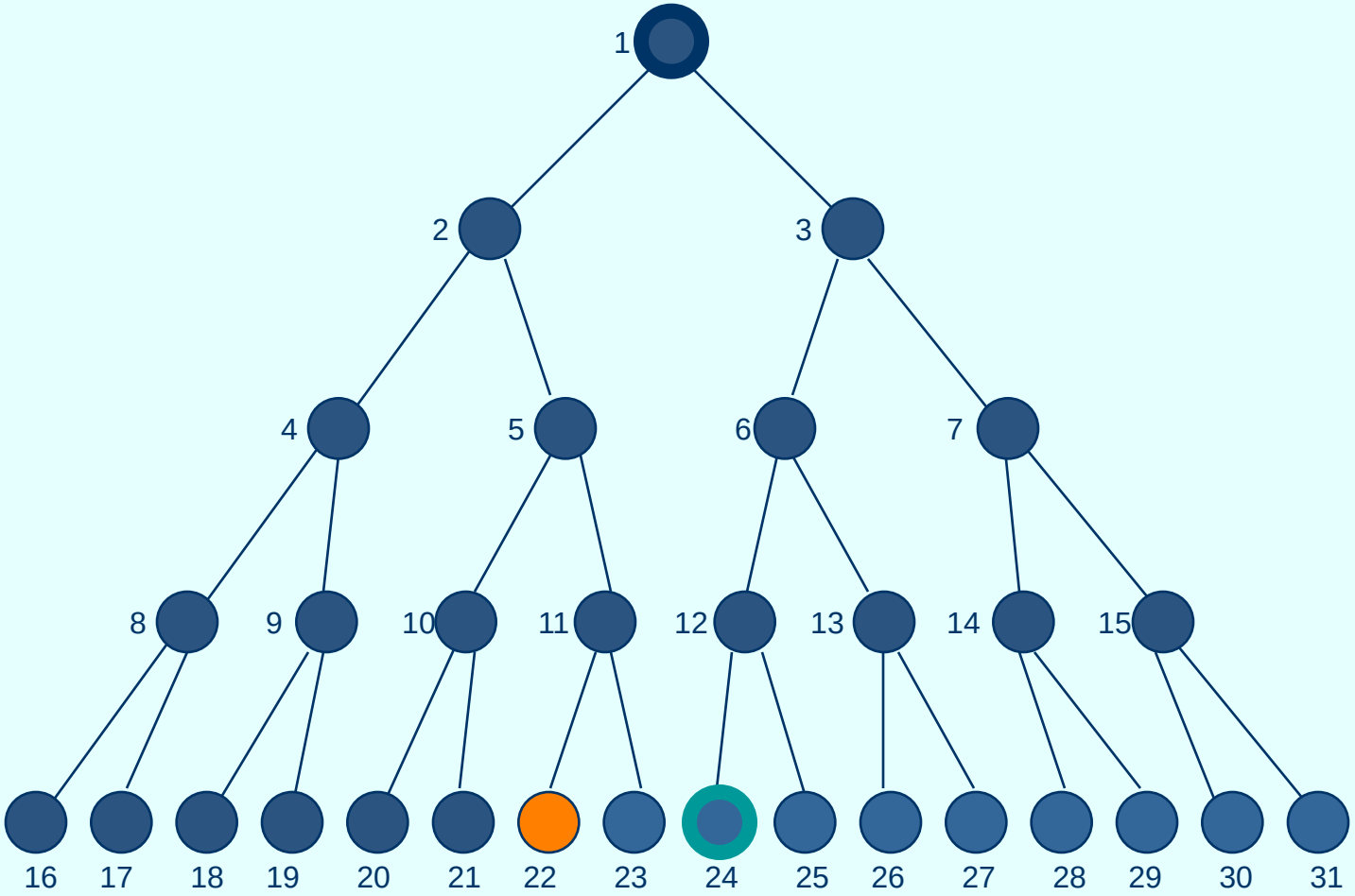
Breadth-First Snapshot 21



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [22,23,24,25,26,27,28,29,30,31]

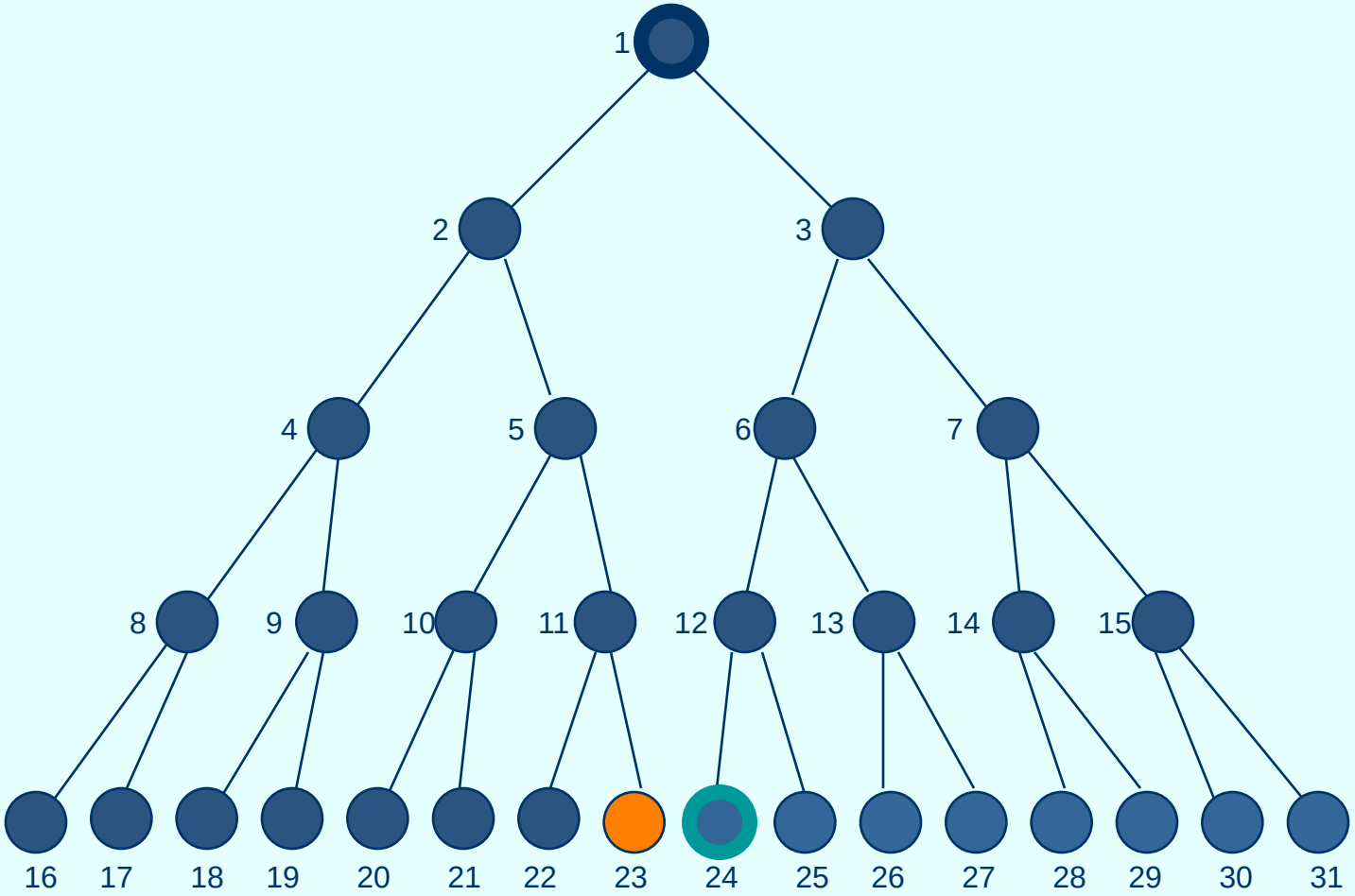
Breadth-First Snapshot 22



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [23,24,25,26,27,28,29,30,31]

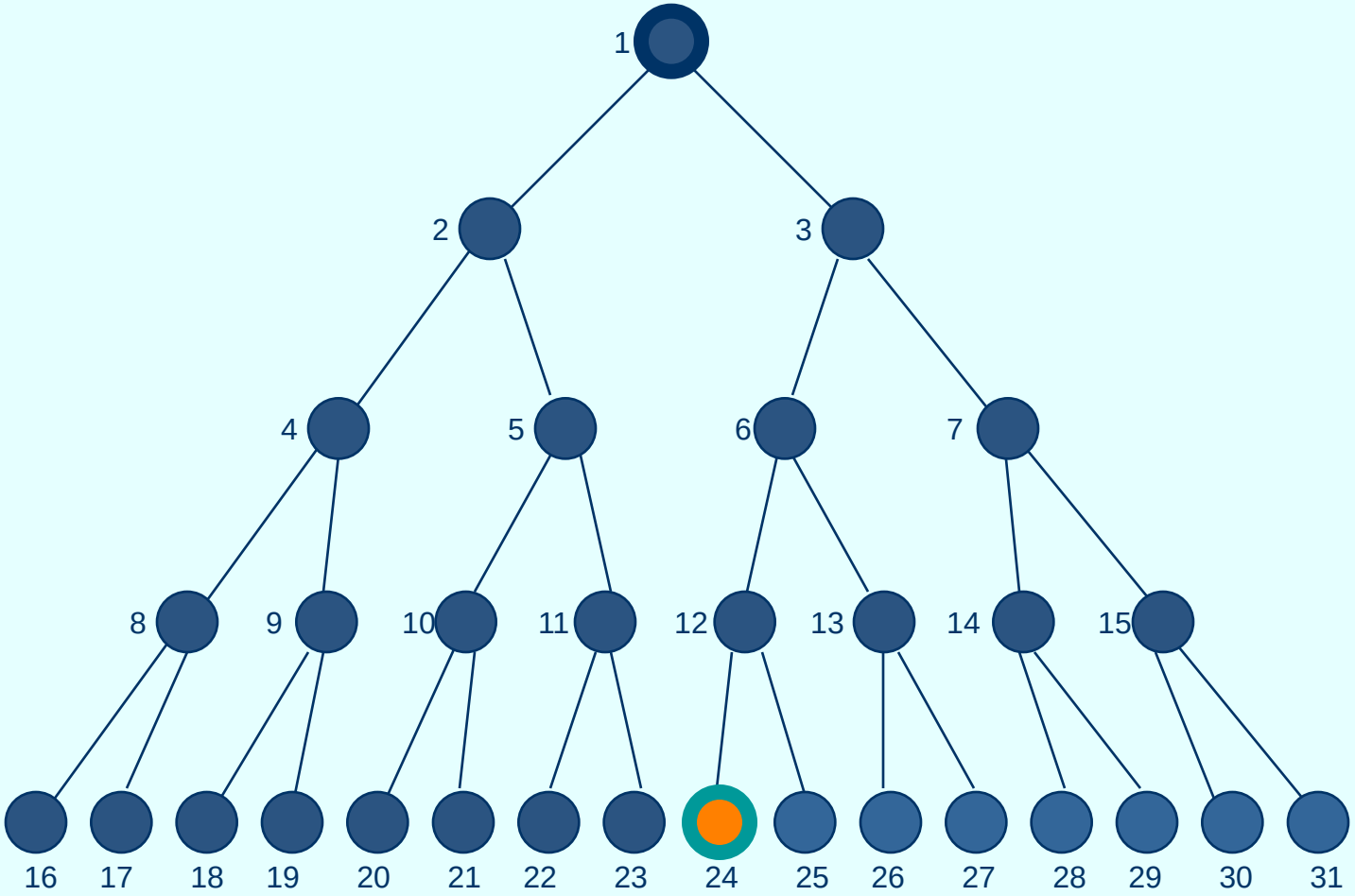
Breadth-First Snapshot 23



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [24,25,26,27,28,29,30,31]

Breadth-First Snapshot 24

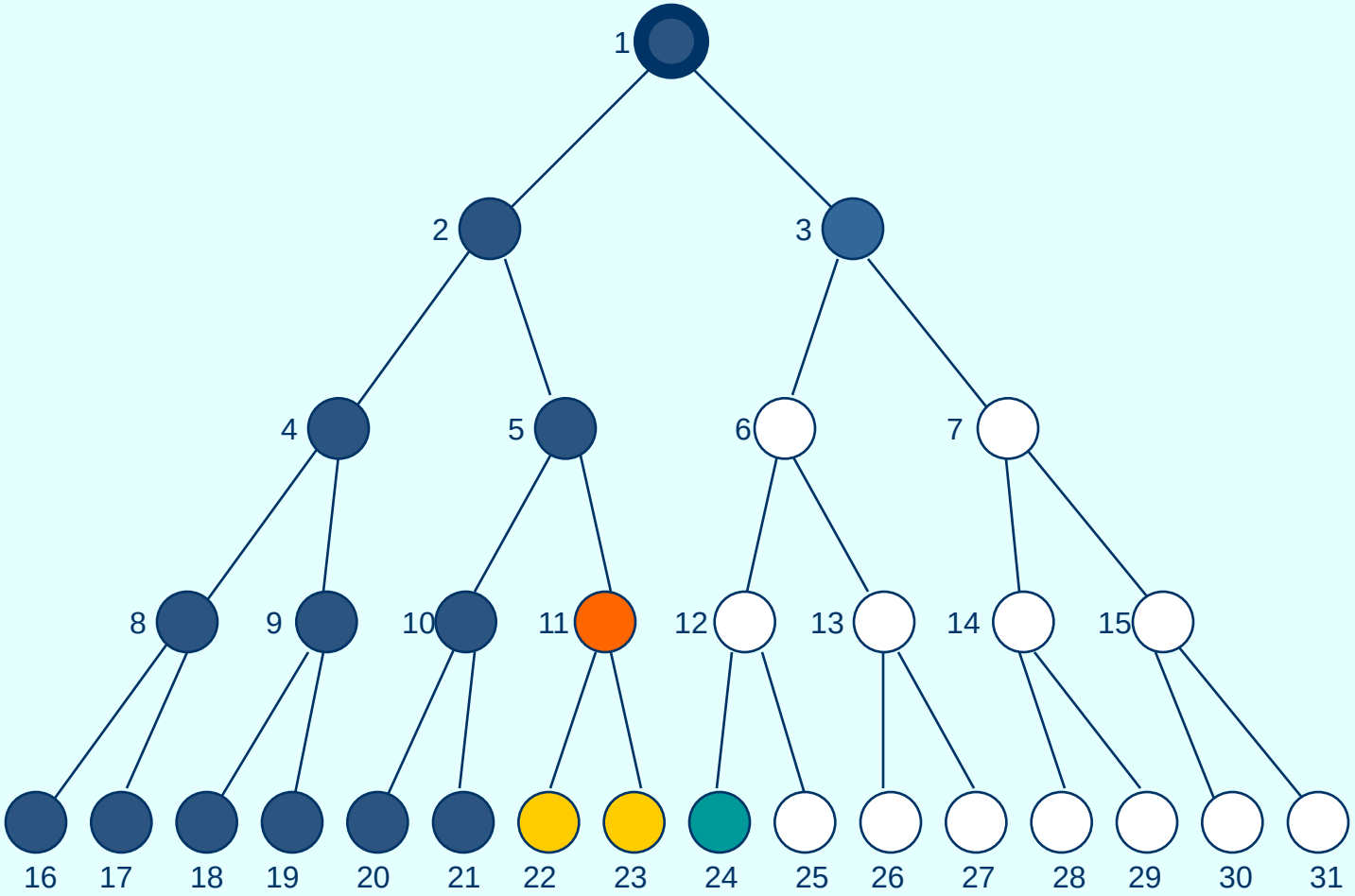


Initial
Visited
Fringe
Current
Visible
Goal

Note:
The goal test is positive for this node, and a solution is found in 24 steps.

Fringe: [25,26,27,28,29,30,31]

Depth-First Snapshot



Initial
Visited
Fringe
Current
Visible
Goal

Fringe: [3] + [22,23]

Depth-First vs. Breadth-First

- depth-first goes off into one branch until it reaches a leaf node
  - not good if the goal is on another branch
  - neither complete nor optimal
  - uses much less space than breadth-first
    - much fewer visited nodes to keep track of
    - smaller fringe
- breadth-first is more careful by checking all alternatives
  - complete and optimal
    - under most circumstances
  - BUT ..very memory-intensive !!

Using State Space to Represent Reasoning..with predicate calculus

- Predicate calculus – map nodes/some states of a graph onto state space
- Inference rules – describe the arc between states
- Eg. Is expression a logical consequence of given assertion? Use search to solve
- Guarantee correctness of conclusion derived- a formal proof of integrity of solution

eg. define a graph from propositional calculus- to view the logical relationships.

$q \rightarrow p$

$r \rightarrow p$

$v \rightarrow q$

$s \rightarrow r$

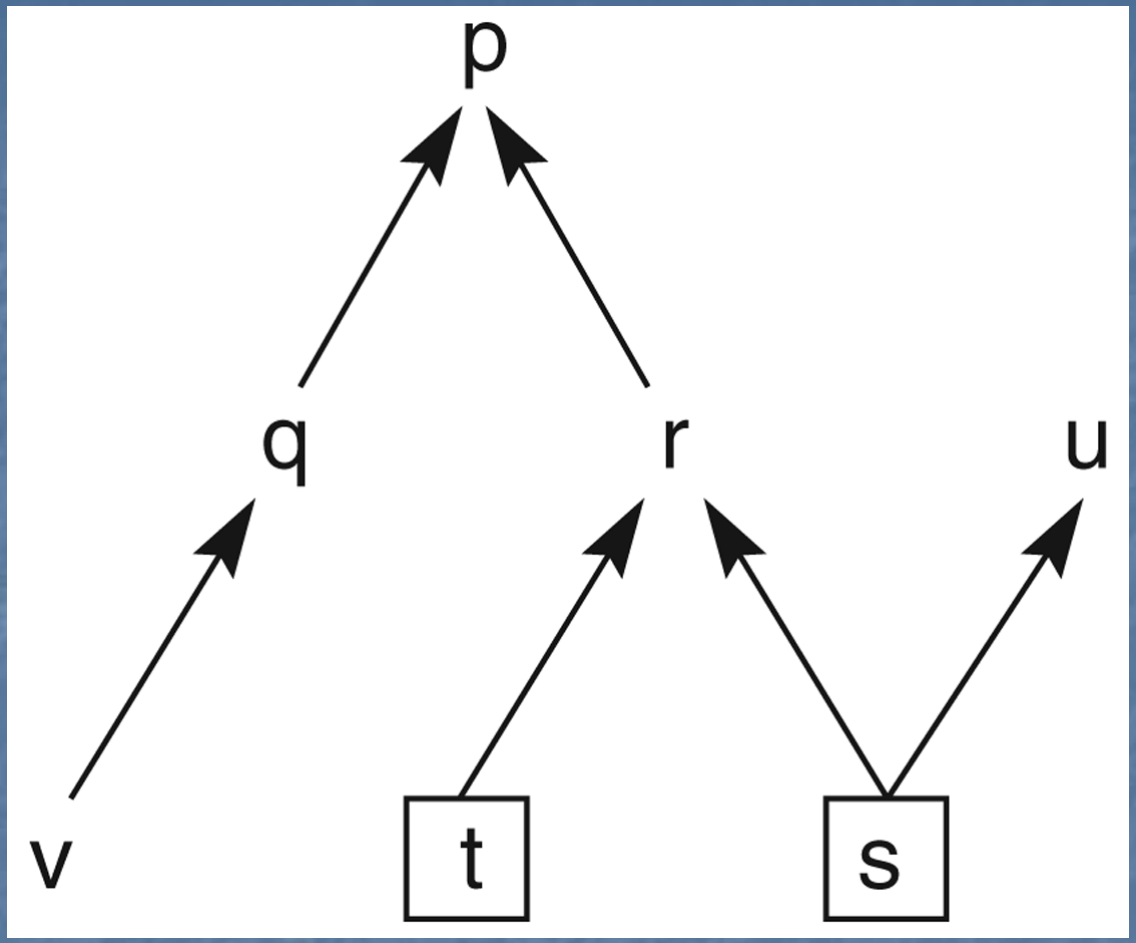
$t \rightarrow r$

$s \rightarrow u$

s

t

Translate into graph??



- State space graph of a set of implications in the propositional calculus.
- Reasoning:- How to infer p using moden ponens?

s , s ->r yields r

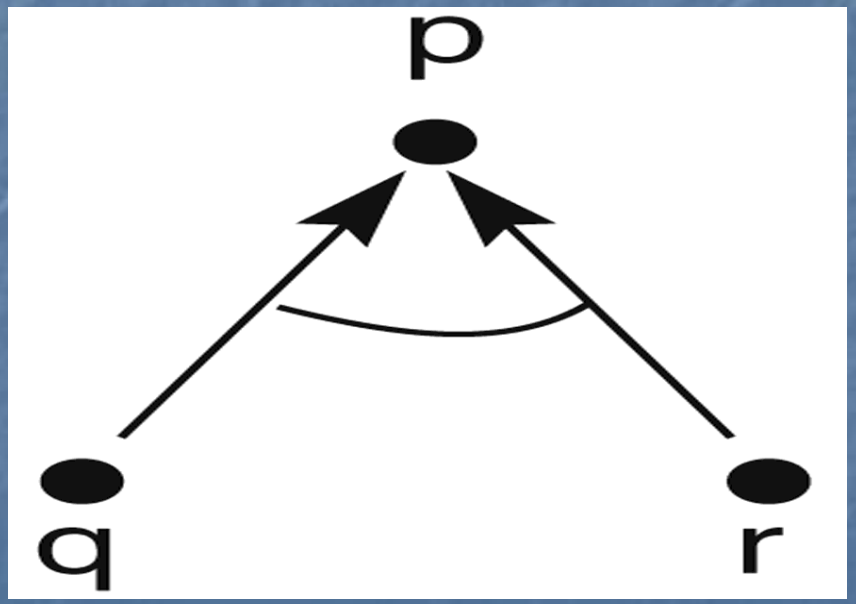
r , r->p yields p

And/or graph

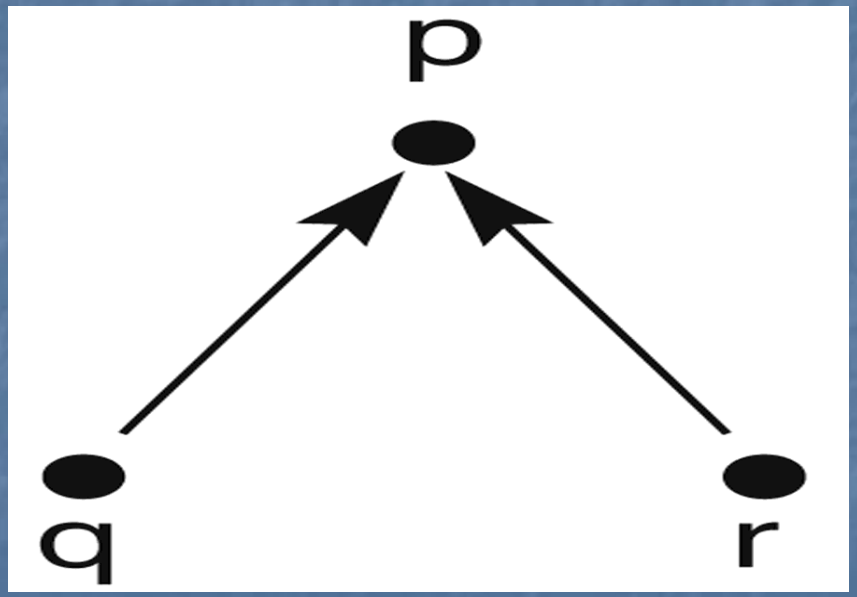
To represent logical operators AND and OR in predic.calculus

- extension of basic graph model known as and/or graph

- To differ the relationships graphically, curve link indicates operator AND.



q $\wedge$ r $\Rightarrow$ p



q $\vee$ r $\Rightarrow$ p

HYPERGRAPH

AND/OR graph is a specialization of a type of graph known as *hypergraph,* which connects nodes by sets of arcs rather than a single arcs

DEFINITION

HYPERGRAPH

A hypergraph consists of:

N, a set of nodes.

H, a set of hyperarcs defined by ordered pairs in which the first element of the pair is a single node from **N** and the second element is a subset of **N**.

An ordinary graph is a special case of hypergraph in which all the sets of descendant nodes have a cardinality of 1.

eg. And/or graph of a set of propositional calculus expressions.

a

b

c

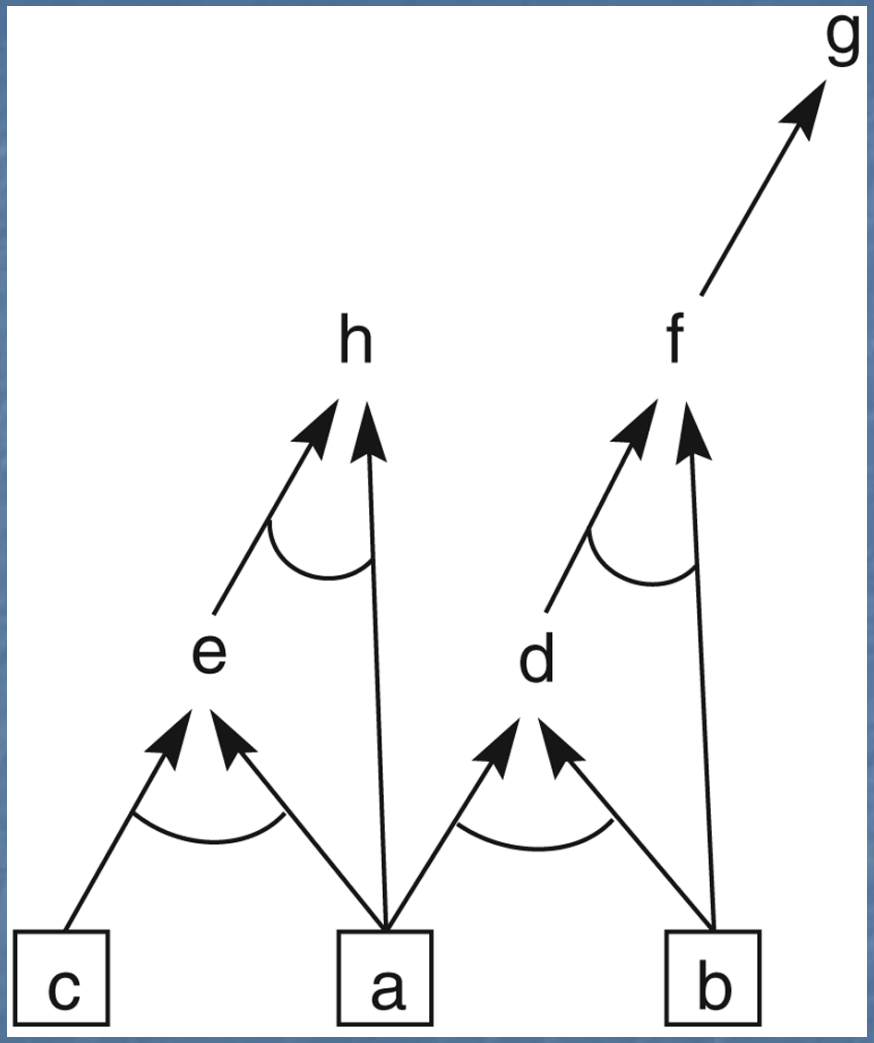
$a \wedge b \rightarrow d$

$a \wedge c \rightarrow e$

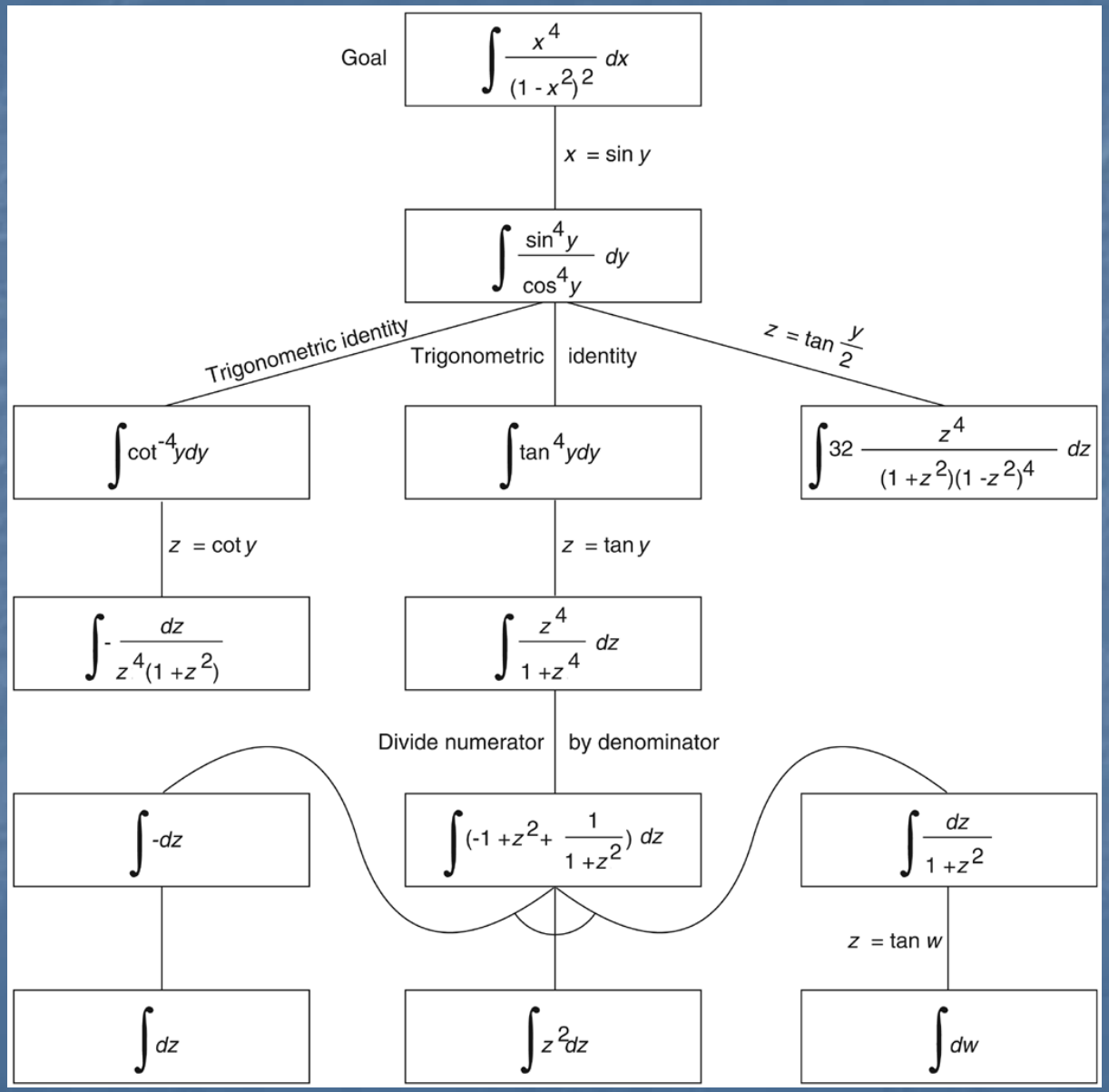
$b \wedge d \rightarrow f$

$f \rightarrow g$

Draw the and/or graph



More eg…And/or graph of part of the state space for integrating a function, from Nilsson (1971).



More eg…Where is Fred??..The facts and rules of this example are given as English sentences followed by their predicate calculus equivalents:

1. Fred is a collie.
   collie(fred).

2. Sam is Fred's master.
   master(fred,sam).

3. The day is Saturday.
   day(saturday).

4. It is cold on Saturday.
   ¬ (warm(saturday)).

5. Fred is trained.
   trained(fred).

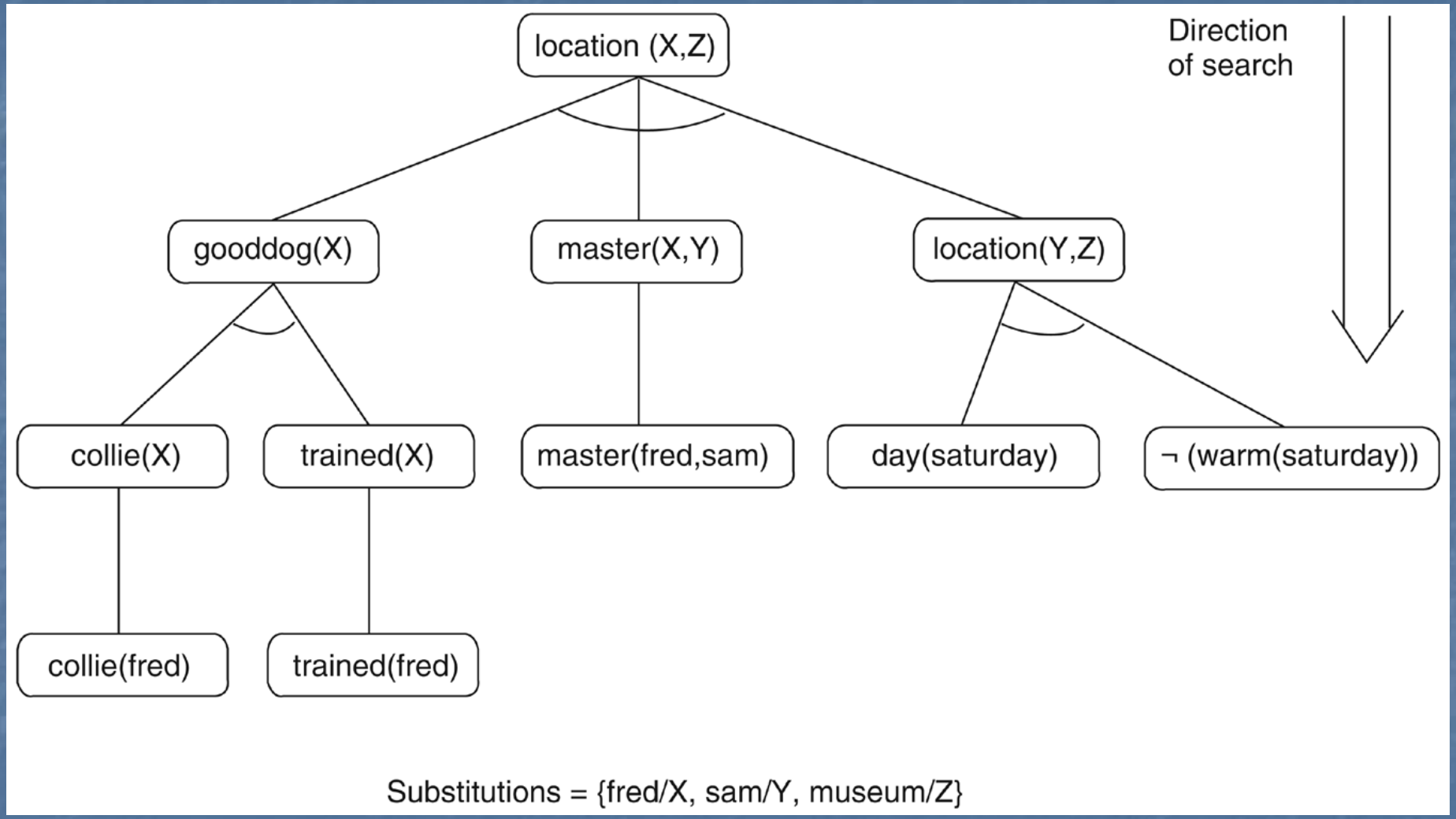
6. Spaniels are good dogs and so are trained collies.
   ∀ X[spaniel(X) ∨ (collie(X) ∧ trained(X)) → gooddog(X)]

7. If a dog is a good dog and has a master then he will be with his master.
   ∀ (X,Y,Z) [gooddog(X) ∧ master(X,Y) ∧ location(Y,Z) → location(X,Z)]

8. If it is Saturday and warm, then Sam is at the park.
   (day(saturday) ∧ warm(saturday)) → location(sam,park).

9. If it is Saturday and not warm, then Sam is at the museum.
   (day(saturday) ∧ ¬ (warm(saturday))) → location(sam,museum).

Where is Fred???.. The solution subgraph showing that fred is at the museum.



More eg...FINANCIAL ADVISOR...And/or graph searched by the financial advisor.

